

UNIVERSIDAD DE CALDAS

Facultad de Inteligencia Artificial e Ingenierías
Departamento de Ingeniería de Sistemas y Computación

TacticalGrid

Informe Técnico

Sistemas Inteligentes I

Autores
Jorge Iván Garcia Torres
Daniel Felipe Franco Rincón

Docente
Jairo Iván Vélez Bedoya

Materia
Sistemas Inteligentes I

Entrega
Octubre de 2026

https://github.com/df1903/tactical-grid-ai

Manizales, Caldas – Colombia
2026

Resumen

TacticalGrid es un entorno de experimentación para algoritmos de inteligencia artificial construido sobre un juego táctico 2D por turnos. Dos bandos (A y B) compiten por un recurso ubicado en zona neutral: una unidad debe recogerlo y llevarlo a su propia base antes que el adversario. El juego no es un producto comercial; su propósito es servir de bancada de pruebas para siete algoritmos fundamentales de IA: **BFS, DFS, UCS, A*, Beam Search, Minimax y Alpha-Beta**.

El proyecto implementa **todos los requisitos obligatorios** del enunciado: modelo de estado compuesto inmutable, carga atómica de escenarios JSON con 9+ validaciones, siete algoritmos con sus métricas (RF-15, RF-19, RF-22, RF-30), dos heurísticas admisibles (Manhattan escalada y obstacle-aware BFS), función de utilidad con pesos configurables en JSON, verificación de soluciones (6 checks RF-42), CLI con sweeps combinatorios y exportación CSV/JSON, GUI tkinter con modo automático IA vs IA, y 10 casos de demostración con resultados esperado vs. obtenido.

Estado: Documento técnico final del proyecto. 281 tests pasan, `ruff` + `mypy -strict` en verde. Código en https://github.com/df1903/tactical-grid-ai.

Índice general

Resumen **1**

Glosario y Abreviaturas **9**

1. Introducción **1**
1.1. Contexto y Motivación . . . 1
1.2. Pregunta Central . . . 1
1.3. Equipo y Entrega . . . 2
1.4. Estructura del Documento . . . 2

2. Modelado del Problema y Representación de Estados **3**
2.1. Principios de Diseño . . . 3
2.2. Partes Estáticas: `Board` . . . 3
2.3. Parte Dinámica: `GameState` . . . 4
2.4. Posición y Vecindad . . . 4
2.5. Caso 6: Estado Compuesto Demostrado . . . 5
2.6. Identificación Unívoca (RF-12) . . . 5
2.7. Justificación de Suficiencia (RF-13) . . . 5

3. Reglas del Juego **7**
3.1. Movimiento y Costos . . . 7
3.2. Ocupación y Bases . . . 7
3.3. Recurso e Interceptación (Placaje) . . . 7
3.4. Fin de Partida . . . 8
3.5. Unidades . . . 8
3.6. Búsqueda de Un Agente (RF-14) . . . 8
3.7. Orden de Sucesores y Desempates . . . 8

4. Costos y Función de Costo **9**
4.1. Definición de Costos (RF-03, RF-04) . . . 9
4.2. Semántica del Costo . . . 9
4.3. Escala de Heurísticas (Sección 18.4 del Enunciado) . . . 9
4.4. Caso 2: Ruta Corta vs. Ruta Económica (RF-04) . . . 10
4.5. Costo en Búsqueda de Un Agente (F2) . . . 10
4.6. Resumen de Reglas de Costo . . . 10

5. Arquitectura del Sistema **11**
5.1. Visión General . . . 11
5.2. Componentes . . . 12
5.3. Comunicación entre Componentes . . . 12
5.4. Estructura de Carpetas . . . 12
5.5. Decisiones Arquitectónicas Clave . . . 13

6. Algoritmos de Búsqueda de Un Agente **14**
6.1. Abstracción de Un Agente: `PathProblem` . . . 14
6.2. Convenciones Comunes (G7, G8, Decisión 2026-10-14) . . . 15
6.3. BFS: Búsqueda en Anchura . . . 15
6.4. DFS: Búsqueda en Profundidad . . . 15
6.5. UCS: Costo Uniforme . . . 16
6.6. Métricas Reportadas (RF-15, RF-16, RF-19, RF-38) . . . 17

7. Heurísticas para A* y Beam Search **18**
7.1. Requisitos (RF-21, RF-22) . . . 18
7.2. h1: Manhattan Escalada . . . 18
7.3. h2: Obstacle-Aware (BFS Inverso desde la Meta) . . . 19
7.4. Propiedades: h2 Domina a h1 . . . 20
7.5. Medición de Efecto (Casos 3 y 4) . . . 20
7.6. Origen de h2 . . . 20

8. Búsqueda Adversarial: Minimax y Alfa-Beta **21**
8.1. Cambio de Paradigma: De Búsqueda a Decisión . . . 21
8.2. Profundidad y Ply (G11, Decisión 2026-10-17) . . . 21
8.3. Función de Utilidad (RF-27, RF-33) . . . 21
8.4. Minimax y Alfa-Beta: Recursión Compartida . . . 22
8.5. Métricas RF-30 (Implementadas en `AdversarialResult`) . . . 23
8.6. RF-31: Misma Decisión con Mismo Orden . . . 23
8.7. RF-32: Orden de Acciones y Poda . . . 23
8.8. Check 6 del Verificador (RF-42) . . . 24
8.9. Casos 7–10: Resultados . . . 24

9. Función de Utilidad: Detalle y Análisis **25**
9.1. Formulación Completa . . . 25
9.2. Justificación de Cada Término . . . 26
9.3. Objetivo Dinámico (Distancia al Objetivo) . . . 26
9.4. Termino Rival: Solo Cuando MAX Porta . . . 26
9.5. Pesos Configurables en JSON (RNF-03) . . . 27
9.6. Limite Conocida: Recurso en Celda Propia vs. Llevarlo Lejos . . . 27
9.7. Explicación para Sustentación: Ejemplo Caso 7 . . . 27

10. Metodología Experimental y Resultados **29**
10.1. Metodología . . . 29
10.2. Casos 1 y 2: No Informada y Costos (F2) . . . 30
10.2.1. Caso 1: BFS vs DFS (Mismo costo, distinta exploración) . . . 30
10.2.2. Caso 2: Ruta Corta vs. Ruta Económica . . . 30

10.3. Casos 3, 4, 5: Informada y Memoria Limitada (F3) 31
10.3.1. Caso 3: Obstáculo y Heurística 31
10.3.2. Caso 4: Comparación de Heurísticas 31
10.3.3. Caso 5: Beam Search y k 31
10.4. Casos 6–10: Adversarial (F4) 32
10.5. Rendimiento (G13) 32

11. Resultados Completos: Tablas Esperado vs. Obtenido **33**
11.1. Caso 1: BFS vs DFS 33
11.2. Caso 2: BFS vs UCS 33
11.3. Caso 3: Obstáculo y Heurística 33
11.4. Caso 4: Comparación de Heurísticas 34
11.5. Caso 5: Beam Search k=1,2,4,8 34
11.6. Caso 6: Estado Compuesto 34
11.7. Caso 7: Decisión Adversarial 34
11.8. Caso 8: Minimax vs Alfa-Beta 35
11.9. Caso 9: Orden de Acciones 35
11.10Caso 10: Profundidad 35

12. Verificación, Testing y Calidad **36**
12.1. Verificación de Soluciones (RF-42) 36
12.2. Testing Automatizado 37
12.3. Cobertura de Requisitos por Test 38
12.4. Calidad de Código 38
12.5. Integración Continua (Manual) 38

13. Casos Mínimos de Demostración: Detalle Completo **39**
13.1. Caso 1: BFS frente a DFS 41
13.2. Caso 2: Ruta Corta vs. Económica 42
13.3. Caso 3: Obstáculo y Heurística 42
13.4. Caso 4: Comparación de Heurísticas 43
13.5. Caso 5: Beam Search 43
13.6. Caso 6: Estado Compuesto 43
13.7. Caso 7: Decisión Adversarial 44
13.8. Caso 8: Minimax vs Alfa-Beta 44
13.9. Caso 9: Orden de Acciones 45
13.10Caso 10: Profundidad 45

14. Formato JSON de Escenarios y Validación Atómica **46**
14.1. Contrato de Interoperabilidad 46
14.2. Campos Obligatorios (RF-35) 47
14.3. Normalización y Tolerancia 48
14.4. Ejemplo Oficial (Apéndice A) 48
14.5. Validación Atómica (RF-34, RF-35) 49
14.6. 9 Validaciones Mínimas (RF-35) + 1 Extra 49
14.7. Campos de `prueba` (G10, Decisión 2026-10-17) 50
14.8. Ejemplos de Escenarios del Equipo 51

15. Limitaciones y Trabajo Futuro **52**
15.1. Limitaciones Conocidas . . . 52
15.2. Trabajo Futuro . . . 53

16. Reporte de Uso de IA Generativa **54**
16.1. Herramienta . . . 54
16.2. Propósito y Áreas de Uso . . . 54
16.3. Intervenciones Humanas y Correcciones . . . 55

17. Reproducibilidad, Versiones y Entrega **57**
17.1. Instalación y Ejecución . . . 57
17.2. Versiones Fijas (RE-02, RE-03, RE-04) . . . 57
17.3. Estados Iniciales Declarados . . . 58
17.4. Resultados Esperados vs. Obtenidos . . . 58
17.5. Comandos de Reproducción (Evidencia RE-03, RE-04) . . . 59
17.6. Identificación de la Entrega (RE-02) . . . 60
17.7. Checklist Final de Entrega . . . 60

18. Conclusiones **61**
18.1. Resumen de Logros . . . 61
18.2. Aportes Destacados . . . 61
18.3. Lecciones Aprendidas . . . 62
18.4. Trabajo Futuro . . . 62
18.5. Agradecimientos . . . 63

A. Diagramas Mermaid → TikZ **64**

B. Código Fuente Completo (Listings) **67**
B.1. Módulo `domain/rules.py` . . . 67
B.2. Módulo `algorithms/informed.py` . . . 71
B.3. Módulo `algorithms/adversarial.py` . . . 75
B.4. Módulo `algorithms/utility.py` . . . 79
B.5. Módulo `scenario/loader.py` . . . 83
B.6. Módulo `experiments/verifier.py` . . . 93

C. Resultados Completos (CSV/JSON) **99**

Índice de figuras

5.1. Arquitectura de componentes y flujo de datos 11

10.1. Ejecución de una búsqueda individual con métricas y verificaciones RF-42. 29
10.2. Sweep de algoritmos con exportación de resultados en CSV y JSON. . . 30

12.1. Ejecución de pytest, ruff y mypy con resultados satisfactorios. 37

13.1. Vista principal de la interfaz gráfica con el escenario cargado. 39
13.2. Controles de algoritmo y parámetros de ejecución. 40
13.3. Resultados de una ejecución mostrados en la interfaz gráfica. 40
13.4. Traza e historial de acciones de una partida. 41
13.5. Interfaz durante una partida automática IA contra IA. 41

14.1. Archivo JSON de escenario utilizado por la aplicación. 46
14.2. Validación de un escenario JSON. 47

A.1. Arquitectura de componentes y flujo de datos. 64
A.2. Ciclo de turno y condiciones de victoria. 65
A.3. Flujo de aplicación de una acción. 66
A.4. Flujo de aplicación con interceptación. 66
A.5. Cronograma de progreso del proyecto. 66

Índice de cuadros

1.1. Información del proyecto 2

2.1. Caso 6: mismo posición, estados distintos 5
2.2. Identificación unívoca de elementos del problema (RF-12) 5

4.1. Caso 2: BFS vs UCS – ruta corta vs. ruta económica 10
4.2. Resumen de reglas de costo 10

5.1. Componentes del sistema y responsabilidades 12
5.2. Comunicación entre componentes 12
5.3. Decisiones arquitectónicas registradas en `decisions.md` 13

6.1. Convenciones comunes de búsqueda (G7, G8, decisión 2026-10-14) . . . 15
6.2. Métricas de búsqueda (Apéndice B + tiempo) 17

7.1. Justificación RF-22 para h1 (Manhattan) 19
7.2. Justificación RF-22 para h2 (Obstacle-aware) 20
7.3. Expandidos en Caso 3 (muro) y Caso 4 (pantano vs. camino abierto) . 20

8.1. Pesos de utilidad (RNF-03: configurables en JSON) 22
8.2. Métricas adversariales (RF-30) 23
8.3. Casos 7–10: Resultados Minimax vs Alfa-Beta 24

9.1. Justificación de términos de la utilidad 26
9.2. Objetivo de la distancia según portador 26

10.1. Caso 1: BFS vs DFS 30
10.2. Caso 2: BFS vs UCS 30
10.3. Caso 3: Expansiones – h2 ve el muro, h1 no 31
10.4. Caso 4: A* con h1 vs h2 31
10.5. Caso 5: Beam Search con k variable 32
10.6. Resumen casos 6–10 32
10.7. Rendimiento Minimax vs Alfa-Beta (15×15, 3 unidades/bando) 32

11.1. Caso 1 – BFS vs DFS (mapa 20×20, todo coste 1, A1 (2,2) → (2,6)) . 33
11.2. Caso 2 – BFS vs UCS (banda pantano coste 7, A1 (5,5) → (5,12)) . . . 33
11.3. Caso 3 – Muro vertical, A* h1 vs h2 (A1 (5,5) → (5,12), muro col 8) . 33
11.4. Caso 4 – Muro con hueco pantano vs. libre (A1 (9,5) → (9,13)) 34
11.5. Caso 5 – Beam Search k=1,2,4,8 (7×9, óptimo costo 28, 14 movs) . . . 34
11.6. Caso 6 – Misma posición (1,0), estados distintos 34

11.7. Caso 7 – Minimax profundidades 1–3 (mapa 5×7, recurso (0,1)) 34
11.8. Caso 8 – Profundidad 4, orden default 35
11.9. Caso 9 – Profundidad 3, orden default vs reversed 35
11.10Caso 10 – Alfa-beta default order, profundidades 1–5 35

12.1. 6 Checks de verificación (RF-42) 36
12.2. Suite de tests (281 tests, 0 skips) 37
12.3. Cobertura de RF por archivo de test 38

13.1. Caso 1 – BFS vs DFS 42
13.2. Caso 2 – BFS vs UCS 42
13.3. Caso 3 – UCS, A* h1, A* h2 42
13.4. Caso 4 – UCS, A* h1, A* h2 43
13.5. Caso 5 – Beam Search k=1,2,4,8 43
13.6. Caso 6 – Estado compuesto 44
13.7. Caso 7 – Minimax profundidades 1–3 44
13.8. Caso 8 – Profundidad 4, orden default 44
13.9. Caso 9 – Profundidad 3, default vs reversed 45
13.10Caso 10 – Alfa-beta default, profundidades 1–5 45

14.1. Campos del formato JSON (obligatorios + opcionales del equipo) . . . 48
14.2. 10 validaciones (9 RF-35 + 1 extra del equipo) 50
14.3. Campos opcionales en `prueba` 51

16.1. Uso de IA por área 55

17.1. Versiones fijas de dependencias 57

Glosario y Abreviaturas

Término	Abreviatura	Definición
Búsqueda en Anchura	BFS	Breadth-First Search: explora nivel por nivel, óptima en número de movimientos
Búsqueda en Profundidad	DFS	Depth-First Search: explora una rama hasta el final antes de retroceder
Costo Uniforme	UCS	Uniform-Cost Search: expande el nodo de menor costo acumulado $g(n)$
A Estrella	A*	A*: $f(n) = g(n) + h(n)$, busca óptimo con heurística admisible
Haz Local	Beam	Beam Search: mantiene solo los k mejores candidatos por nivel
Minimax	MM	Algoritmo adversarial: MAX maximiza, MIN minimiza
Alfa-Beta	α-β	Minimax con poda: elimina ramas que no cambian la decisión
RF	RF	Requisito Funcional (del enunciado)
RNF	RNF	Requisito No Funcional
PLY	–	Una acción de un bando (profundidad = número de plies)
MAX/MIN	–	Roles en el árbol Minimax (no agentes distintos)
Utilidad	$U(s)$	Valor numérico de un estado desde la perspectiva de MAX
Heurística Admisible	–	$h(n) \leq h^*(n)$: nunca sobreestima el costo real restante
Heurística Consistente	–	$h(n) \leq c(n, n') + h(n')$: cumple desigualdad triangular
Factor de Ramificación	b	Número promedio de sucesores por estado
Profundidad de Solución	d	Número de movimientos hasta la meta
Estado Compuesto	–	Posiciones + carrier + resource + turn + turn_number

Abreviaturas de archivos:

BFS Breadth-First Search

DFS Depth-First Search

UCS Uniform Cost Search

ASTAR A* (A Estrella)

BEAM Beam Search

MM Minimax

AB Alpha-Beta

CLI Command Line Interface

GUI Graphical User Interface

JSON JavaScript Object Notation

CSV Comma-Separated Values

SVG Scalable Vector Graphics

PDF Portable Document Format

UCALDAS Universidad de Caldas

IA Inteligencia Artificial

Capítulo 1

Introducción

1.1 Contexto y Motivación

TacticalGrid nace como proyecto integrador del curso **Sistemas Inteligentes I** (Universidad de Caldas, 2026-2). El trabajo plantea la construcción de un juego táctico 2D por turnos que sirva como *entorno de experimentación* para siete algoritmos clásicos de inteligencia artificial:

1. **BFS** (Breadth-First Search) – búsqueda en anchura
2. **DFS** (Depth-First Search) – búsqueda en profundidad
3. **UCS** (Uniform-Cost Search) – costo uniforme
4. **A*** – A* con al menos dos heurísticas admisibles
5. **Beam Search** – búsqueda con memoria limitada ($k = 1, 2, 4, 8$)
6. **Minimax** – decisión adversarial con profundidad limitada
7. **Alpha-Beta** – Minimax con poda alfa-beta

La premisa central del enunciado es clara: *"El juego es el entorno de experimentación. El proyecto es el sistema inteligente que decide dentro de él."* No se busca un videojuego comercial; la interfaz solo debe permitir **observar y experimentar con la inteligencia** del sistema.

1.2 Pregunta Central

El enunciado plantea la pregunta guía del proyecto:

> *¿Cómo cambia la forma de decidir de un sistema inteligente cuando pasa de encontrar una ruta, a optimizar su costo, utilizar conocimiento heurístico y finalmente actuar frente a un adversario que también toma decisiones?*

Esta pregunta estructura las cuatro etapas del proyecto:

1. **Encontrar una ruta** → BFS, DFS

2. **Optimizar el costo** → UCS
3. **Usar conocimiento heurístico** → A*, Beam Search
4. **Decidir frente a adversario** → Minimax, Alpha-Beta

1.3 Equipo y Entrega

Cuadro 1.1: *Información del proyecto*

Integrantes	Jorge Iván Garcia Torres, Daniel Felipe Franco Rincón
Materia	Sistemas Inteligentes I – Ingeniería de Sistemas y Computación
Universidad	Universidad de Caldas (UCALDAS)
Entrega	Octubre de 2026
Repositorio	`https://github.com/df1903/tactical-grid-ai`
Stack	Python 3.14, `uv`, `pytest`, `ruff`, `mypy -strict`

1.4 Estructura del Documento

Este informe técnico sigue la estructura del enunciado y cubre:

1. **Modelado y estado** (Cap. 2) – Estado compuesto, reglas, RF-10 a RF-13
2. **Reglas y costos** (Cap. 3, 4) – Reglas deterministas, RF-09
3. **Arquitectura** (Cap. 5) – 5 componentes desacoplados (RNF-05)
4. **Algoritmos de búsqueda** (Cap. 6 – 7) – BFS, DFS, UCS, A*, Beam
5. **Búsqueda adversarial** (Cap. 8) – Minimax, α-β, utilidad
6. **Experimentación** (Cap. 10) – Metodología, casos 1-10, rendimiento
7. **Verificación y testing** (Cap. 12) – 6 checks RF-42, 281 tests
8. **Formato JSON** (Cap. 14) – 9+ validaciones, campos opcionales
9. **Limitaciones** (Cap. 15) – 8 ítems documentados
10. **Uso de IA** (Cap. 16) – Reporte de uso de IA generativa
11. **Reproducibilidad** (Cap. 17) – Instalación, tag Git, comandos

Todos los resultados numéricos (casos 1-10, rendimiento) se derivaron **a mano antes de ejecutar el código** y son verificados por tests automatizados (`tests/test_cases_f*.py`). Los archivos CSV/JSON brutos están en `docs/results/`.

Capítulo 2

Modelado del Problema y Representación de Estados

Este capítulo describe el modelado del problema como espacio de estados (Unidad 2 del curso) y la representación del estado compuesto que soporta tanto las búsquedas de un agente (F2, F3) como la búsqueda adversarial (F4). Cumple con RF-10–RF-13 y el caso 6.

2.1 Principios de Diseño

El modelado sigue tres principios fundamentales:

1. **Inmutabilidad:** `GameState` y `Board` son `@dataclass(frozen=True)`. La inmutabilidad garantiza que los estados puedan usarse como claves en `dict`/`set` (visited, frontier, parents) sin riesgo de mutación accidental.
2. **Separación estática/dinámica:** `Board` contiene todo lo que **nunca cambia** (grid, catálogo de terrenos, bases, bando de cada unidad, límite de turnos). `GameState` contiene solo lo que **cambia** (posiciones, carrier, resource, turn, turn_number).
3. **Estado compuesto completo (RF-10, RF-11):** Un estado no es solo una posición; dos estados con la misma posición física son distintos si difieren en `carrier`, `resource`, `turn`, `turn_number` o las posiciones de otras unidades. El **caso 6** demuestra esto explícitamente (ver Tabla 2.5).

2.2 Partes Estáticas: `Board`

`Board` es la parte estática del escenario; se construye una sola vez al cargar el JSON y nunca muta. Contiene:

- `rows`, `cols`: dimensiones del grid
- `grid`: tupla de tuplas con nombres de terreno por celda
- `terrains`: `Mapping[str, Terrain]` – catálogo `nombre` → `Terrain(cost, passable)`

- `bases`: `Mapping[Side, Position]` – base de cada bando
- `unit_sides`: `Mapping[str, Side]` – bando de cada unidad (clave: id)
- `turn_limit`: límite de turnos para empate (default 100, configurable en JSON)

Métodos clave:

```
def in_bounds(self, pos: Position) -> bool
def terrain_at(self, pos: Position) -> Terrain
def is_passable(self, pos: Position) -> bool
def cost_at(self, pos: Position) -> float
```

2.3 Parte Dinámica: GameState

`GameState` es el estado compuesto inmutable (frozen dataclass). Contiene:

```
@dataclass(frozen=True)
class GameState:
    positions: tuple[tuple[str, Position], ...]  # ordenado por unit_id
    carrier: str | None          # id de la unidad que lleva el recurso
    resource: Position | None    # celda del recurso si nadie lo lleva
    turn: Side                   # Side.A o Side.B
    turn_number: int = 0         # para límite de turnos / empate
```

Métodos auxiliares:

```
def position_of(self, unit_id: str) -> Position
def unit_at(self, pos: Position) -> str | None
```

2.4 Posición y Vecindad

`Position` es una tupla inmutable `(row, col)` con métodos:

```
def neighbors(self) -> tuple[Position, ...]:  # 4 dirs: arriba, abajo, izq,
↪  der
def is_adjacent(self, other: Position) -> bool  # distancia Manhattan == 1
```

La vecindad es de **4 direcciones** (arriba, abajo, izquierda, derecha) por decisión del equipo (decisión 2026-10-07). Con 4 direcciones, la heurística Manhattan es admisible.

2.5 Caso 6: Estado Compuesto Demostrado

El **caso 6** demuestra RF-11: "dos estados con una unidad en la misma casilla pueden ser distintos". En el mapa del caso 7 (5×7, coste 1), la unidad A1 en (1,0) junto a su base (0,0):

Cuadro 2.1: *Caso 6: mismo posición, estados distintos*

Estado	Pos A1	Carrier	Turn	Efecto de Move A1 → (0,0)
Libre (inicial)	(1,0)	nadie	A	entra a la base, no hay ganador
Portando	(1,0)	A1	A	entra a la base → **gana A**
Turno rival	(1,0)	nadie	B	A1 no tiene acciones legales

Los tres estados tienen `positions` idénticas pero son objetos distintos (`len(set(states)) == 3`). Un diccionario indexado por estado mantiene tres entradas; indexado por posición solo mantendría una. Test: `tests/test_case_6.py`.

2.6 Identificación Unívoca (RF-12)

La Tabla 2.2 resume dónde se encuentra cada elemento requerido por RF-12.

Cuadro 2.2: *Identificación unívoca de elementos del problema (RF-12)*

Elemento	Dónde se define
Estado inicial	`Scenario.initial_state` (construido por `scenario/loader.py`)
Acciones válidas	`domain.rules.legal_actions(board, state)`
Estados sucesores	`domain.rules.successors(board, state)` → `(action, next_state, cost)`
Condición de meta	Búsqueda: `problem.is_goal(state)`; Juego: `winner(board, state)`
Costo de acción	`domain.rules.apply(board, state, action)` → `(next_state, cost)`

2.7 Justificación de Suficiencia (RF-13)

El estado contiene **exactamente** la información necesaria para decidir:

1. **Dónde está cada unidad** (`positions`) → legalidad de movimientos, ocupación
2. **De qué bando es cada unidad** (`unit_sides` en `Board`) → turno, legalidad
3. **Quién lleva el recurso** (`carrier`) → pickup, delivery, interceptación
4. **Dónde está el recurso si nadie lo lleva** (`resource`) → pickup
5. **De quién es el turno** (`turn`) → qué unidades pueden moverse
6. **Número de turno** (`turn_number`) → empate por límite

Nada más influye en las reglas (no hay vida, munición, información oculta). Por tanto, dos estados con iguales valores en estos seis campos tienen idéntico conjunto de acciones legales, mismos sucesores, mismos costos y mismo resultado final. RF-13 satisfecho.

Capítulo 3

Reglas del Juego

Este capítulo describe las reglas deterministas del juego (RF-09), decididas por el equipo y documentadas en `decisions.md` (2026-10-07, 2026-10-14, 2026-10-18).

3.1 Movimiento y Costos

- **Vecindad:** 4 direcciones (arriba, abajo, izquierda, derecha). Decisión 2026-10-07.
- **Costo:** Se paga al **entrar** a la celda destino; la celda inicial no cuesta. Costos y transitabilidad provienen del catálogo `tipos_terreno` del JSON.
- **Wait:** Siempre legal, cuesta 0. No recoge recurso.

3.2 Ocupación y Bases

- **Una unidad por celda:** Validado al cargar (extra a RF-35).
- **Base propia:** Se puede pisar; al llegar con el recurso → victoria inmediata.
- **Base rival: NO** se puede pisar (decisión 2026-10-18).
- **Celda recurso:** Se pisa; recoge al **llegar** (Move), no al esperar.

3.3 Recurso e Interceptación (Placaje)

- **Pickup:** Al **llegar** a la celda del recurso (Move, no Wait).
- **Entrega:** Llegar a base propia con recurso → victoria inmediata (salta interceptación).
- **Interceptación (Placaje):** Tras **cualquier** movimiento, si un rival queda adyacente (distancia Manhattan = 1) al portador → recurso **cae en la celda del portador** (sin transferencia al rival; no hay desempate).
- Portador original **no recoge** mientras siga en esa celda; debe salir y volver.
- Cualquier **otra unidad** (incluso del mismo bando) lo recoge al llegar a la celda.
- Se evalúa tras **cualquier** movimiento (de cualquier bando).

3.4 Fin de Partida

- **Victoria:** Llevar recurso a base propia.
- **Empate:** Límite de turnos (`juego.limite_turnos`, default 100, configurable en JSON).
- Bando sin movimientos legales → pasa turno (`Wait` siempre legal).

3.5 Unidades

- Un solo tipo (`estandar`); mínimo 3 por bando en escenarios propios.
- Apéndice A trae 2 → cargador no exige mínimo al cargar.

3.6 Búsqueda de Un Agente (RF-14)

Para búsquedas (BFS, DFS, UCS, A*, Beam):

- Rivales **removidos** del estado (si no, Apéndice A sin solución: B1 sobre meta).
- Solo unidad inicio se mueve; compañeros quietos como obstáculos.
- Turno fijo al bando de la unidad; recurso ignorado; `Wait` no usado.
- Base rival sigue no entrable (regla de dominio).
- Estado de búsqueda ≡ posición de la unidad.

3.7 Orden de Sucesores y Desempates

- Orden fijo: **arriba, abajo, izquierda, derecha** (método `Position.neighbors()`).
- DFS: hijos en orden **inverso** para expandir primero "arriba".
- UCS/A*: desempate por **orden de entrada a la frontera** (FIFO tie-break).
- Actualización de costo: camino más barato actualiza g y padre, **no** regenera.
- $max_frontier$ = tamaño del set `waiting` (estados distintos esperando).

Capítulo 4

Costos y Función de Costo

4.1 Definición de Costos (RF-03, RF-04)

Los costos **nunca están hardcodeados** en el código (RF-03, RF-37, RF-04). Provienen exclusivamente del catálogo `tipos_terreno` del JSON cargado:

- `tipos_terreno` es un mapeo `nombre` → `{costo, transitable}`
- Solo terrenos `transitable: true` tienen `costo` (positivo, `float`)
- Terrenos no transitables (`transitable: false`) tienen `costo: null`
- Acepta enteros o decimales (G12)

4.2 Semántica del Costo

- El costo se paga **al entrar** a la celda destino (decisión 2026-10-07).
- La celda inicial **no cuesta**.
- `Wait` cuesta `0.0` (decisión 2026-10-14).
- No hay otras acciones con costo (solo `Move` y `Wait`).

Esto implica que $g(n)$ es la suma de los costos de las celdas **destino** de cada movimiento en el camino. La celda inicial nunca contribuye a $g(n)$.

4.3 Escala de Heurísticas (Sección 18.4 del Enunciado)

Dado que los costos vienen del JSON y un mapa externo podría traer costos < 1, la heurística Manhattan "pura" (1 por paso) dejaría de ser admisible si mín(costos) < 1.

La solución adoptada: **escalar por el costo transitable mínimo del escenario cargado**:

```python
def min_step_cost(board: Board) -> float:
    costs = [t.cost for t in board.terrains.values() if t.passable and
    ↪  t.cost is not None]
    return min(costs)
```

Ambas heurísticas (h_1 Manhattan, h_2 obstacle-aware) multiplican su estimación en pasos por `min_step_cost()`. Esto garantiza admisibilidad para **cualquier** JSON válido, como se muestra en la definición anterior.

4.4 Caso 2: Ruta Corta vs. Ruta Económica (RF-04)

El enunciado exige que existan mapas donde una ruta con menos movimientos cueste más que una ruta más larga. **Caso 2** (`caso_02_bfs_vs_ucs.json`) lo demuestra:

- Banda de pantano (costo 7) bloquea el camino directo
- BFS cruza el pantano: 7 movimientos, costo $4 \times 1 + 3 \times 7 = 25$
- UCS rodea por terreno barato: 17 movimientos, costo 17

Cuadro 4.1: *Caso 2: BFS vs UCS – ruta corta vs. ruta económica*

Algoritmo	Costo	Movimientos	Generados	Expandidos	Max Frontera
BFS	25	7	126	104	23
UCS	17	17	269	239	33

4.5 Costo en Búsqueda de Un Agente (F2)

En la abstracción de un agente (RF-14), el problema de búsqueda usa `PathProblem` donde `expand` devuelve `[(GameState, cost)]` llamando a `domain.rules.successors` y filtrando solo `Move` de la unidad inicio. El costo devuelto es `board.cost_at(dest)`.

4.6 Resumen de Reglas de Costo

Cuadro 4.2: *Resumen de reglas de costo*

Regla	**Valor**
Costo de Move	`board.cost_at(dest)` (terreno destino)
Costo de Wait	0.0
Celda inicial	No cuesta
Costos vienen de	`tipos_terreno` del JSON (RF-03)
No hay costos hardcodeados	(RNF-03, RNF-04)

Capítulo 5

Arquitectura del Sistema

5.1 Visión General

La arquitectura sigue el principio de **separación de responsabilidades** (RNF-05): cinco componentes desacoplados que se comunican mediante interfaces bien definidas. Ningún componente conoce detalles de implementación de otro; solo dependen de interfaces (clases, funciones, tipos de datos).

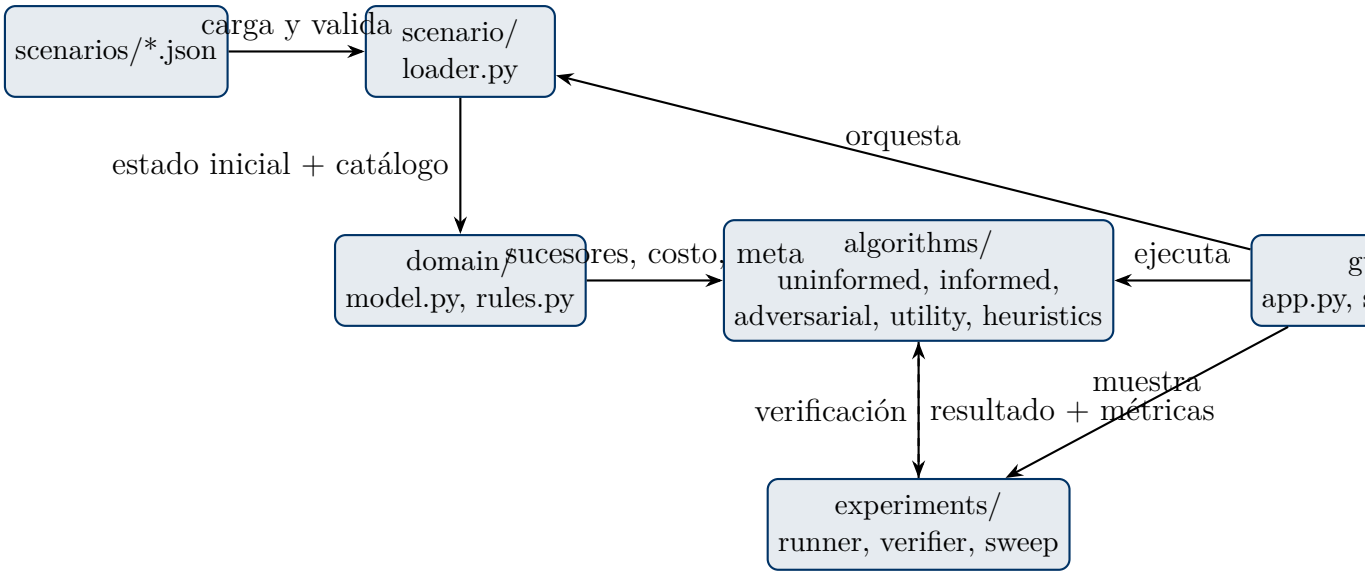


Figura 5.1: *Arquitectura de componentes y flujo de datos*

5.2 Componentes

Cuadro 5.1: *Componentes del sistema y responsabilidades*

Componente	Archivos clave
`domain`	`model.py`, `rules.py`
`scenario`	`loader.py`
`algorithms`	`common.py`, `uninformed.py`, `informed.py`, `adversarial.py`, `utility.py`, `heuristics.py`
`experiments`	`runner.py`, `verifier.py`, `sweep.py`
`gui`	`app.py`, `session.py`

5.3 Comunicación entre Componentes

Cuadro 5.2: *Comunicación entre componentes*

De	Hacia	Qué se comunica
`scenario`	`domain`	Estado inicial `GameState` + `Board` (catálogo validado)
`algorithms`	`domain`	Consulta `legal_actions`, `successors`, `apply`, `winner`
`experiments`	`algorithms`	Ejecuta algoritmos, recibe `SearchResult`/`AdversarialResult`
`gui`	`scenario`, `algorithms`, `experiments`	Orquesta y muestra; **no contiene lógica de juego**

Principios clave:

- `algorithms` **nunca lee JSON** ni conoce la GUI; solo usa `domain` y `Scenario`.
- `gui` no contiene lógica de juego; llama a `domain` (movimientos), `algorithms` (decisiones) y `experiments` (búsquedas).
- `scenario` valida **atómicamente**: todo o nada, errores explicativos, nada parcialmente construido.

5.4 Estructura de Carpetas

```
1 src/tactical_grid/
```

```
 domain/          estado, acciones, sucesores, costos, reglas
 scenario/        carga JSON, validación atómica
 algorithms/      BFS, DFS, UCS, A*, Beam, Minimax, -, heurísticas, utilidad
 gui/             interfaz gráfica tkinter
 experiments/     métricas, verificador, casos mínimos, runner, sweep

scenarios/           JSON de escenarios (apendice_a.json, caso_01..07,
↪  prueba_15x15, etc.)
tests/               281 pruebas automatizadas
docs/                enunciado, informe técnico, manual, casos, resultados
latex/               este documento (LaTeX)
```

5.5 Decisiones Arquitectónicas Clave

Cuadro 5.3: *Decisiones arquitectónicas registradas en* `decisions.md`

Fecha	Decisión
2026-10-07	5 componentes: domain, scenario, algorithms, gui, experiments
2026-10-07	Separación estática/dinámica: Board (estático) + GameState (dinámico)
2026-10-07	Inmutabilidad total: frozen dataclasses, tuplas ordenadas
2026-10-07	JSON claves en español, código en inglés
2026-10-16	GUI: tkinter (estándar, cero dependencias)
2026-10-17	CLI: sweep combinatorio, export CSV/JSON

Capítulo 6

Algoritmos de Búsqueda de Un Agente

Este capítulo cubre las búsquedas de un solo agente (RF-14 a RF-24): BFS, DFS, UCS, A*, Beam Search. El modelo de abstracción (`PathProblem`) transforma el juego completo en un problema de un agente (RF-14).

6.1 Abstracción de Un Agente: `PathProblem`

La RF-14 exige ignorar al adversario. `make_problem` transforma el `GameState` completo en un problema de **un solo agente**:

1. **Quita rivales:** las unidades del bando contrario se eliminan de `positions`.
2. **Compañeros quietos:** las otras unidades propias se quedan quietas como obstáculos.
3. **Turno fijo:** el turno se fija al bando de la unidad inicio; no alterna.
4. **Recurso ignorado:** `carrier=None`, `resource=None`.
5. **Turn number = 0:** se reinicia el contador.
6. **Base rival no entrable:** se mantiene la regla de dominio (no se puede pisar).

```
def make_problem(board: Board, initial: GameState, unit_id: str, goal:
↪  Position) -> PathProblem:
    side = board.unit_sides[unit_id]
    start = replace(
        initial,
        positions=tuple((uid, pos) for uid, pos in initial.positions
                        if board.unit_sides[uid] is side),
        turn=side,
        turn_number=0,
        carrier=None,
        resource=None,
    )
    return PathProblem(board, start, unit_id, goal)
```

El estado de búsqueda equivale a la **posición de la unidad inicio**; los compañeros permanecen como obstáculos estáticos.

6.2 Convenciones Comunes (G7, G8, Decisión 2026-10-14)

Todos los algoritmos de búsqueda (BFS, DFS, UCS, A*, Beam) comparten estas convenciones:

Cuadro 6.1: *Convenciones comunes de búsqueda (G7, G8, decisión 2026-10-14)*

Convención	Valor
Tipo de búsqueda	Grafo (visited set); un estado nunca se genera dos veces
Meta	Probada al **expandir** (no al generar)
Estado inicial	Cuenta como **generado** (cuenta = 1)
Orden de sucesores	Arriba, Abajo, Izquierda, Derecha (orden de `Position.neighbors()`)
DFS	Empuja hijos en orden **inverso** para expandir primero .ªrriba"
Desempate UCS/A*	Orden de entrada a la frontera (FIFO tie-break)
Actualización de costo	UCS/A*: camino más barato actualiza g y padre, **no** regenera
$max_frontier$	Tamaño del conjunto `waiting` (estados distintos esperando)
DFS	Sin límite de profundidad (visited set garantiza terminación)

Justificación: La meta al expandir (no al generar) es necesaria para que UCS/A* garanticen optimalidad. El estado inicial cuenta como generado porque entra a la frontera. El desempate FIFO asegura determinismo y reproduce los resultados esperados de los casos de prueba.

6.3 BFS: Búsqueda en Anchura

```
def bfs(problem: PathProblem) -> SearchResult:
    # FIFO queue (deque)
    # goal test al expandir
    # no duplicados: skip si en explored o en parents (frontera)
    # orden: up, down, left, right
```

- **Frontera:** `deque` (FIFO)
- **Complejidad:** $\mathcal{O}(V)$ tiempo y espacio sobre V estados alcanzables
- **Óptimo en:** número de movimientos (si todos los costos son iguales)

6.4 DFS: Búsqueda en Profundidad

```
def dfs(problem: PathProblem) -> SearchResult:
```

```
    # LIFO stack (list)
    # hijos en orden inverso -> pop() expande primero "arriba"
    # mismo bookkeeping que BFS (grafo, meta al expandir, no duplicados)
```

- **Frontera:** `list` (LIFO)
- **Hijos en orden inverso:** para que `pop()` saque primero "arriba"
- **Complejidad:** $\mathcal{O}(V)$ tiempo y espacio
- **No óptimo** en movimientos ni en costo

6.5 UCS: Costo Uniforme

Implementa la convención exacta de RF-19 (curso): prioridad = menor $g(n)$; desempate por orden de entrada a la frontera; actualización de costo sin regenerar.

```
def ucs(problem: PathProblem) -> SearchResult:
    heap = [(0.0, 0)]            # (g, order_rank)
    order = {start: 0}           # estado -> rank de entrada
    by_order = [start]           # rank -> estado
    waiting = {start}            # estados en frontera (para max_frontier)

    while heap:
        g_cost, rank = heapq.heappop(heap)
        node = by_order[rank]
        if node in explored or g_cost > g[node]: continue  # stale entry
        waiting.discard(node)
        # goal test, expandir, actualizar...

        if child not in g:
            # PRIMERA VEZ
            order[child] = len(by_order)
            by_order.append(child)
            waiting.add(child)
            generated += 1
            heapq.heappush(heap, (new_cost, order[child]))
        elif new_cost < g[child]:
            # CAMINO MÁS BARATO: actualiza g y padre, MANTIENE order
            ↪ original
            g[child] = new_cost
            heapq.heappush(heap, (new_cost, order[child]))  # nuevo entry,
            ↪ mismo rank
```

Puntos clave de la implementación:

1. `order` = orden de primera entrada a la frontera (tie-break RF-19)

2. Actualización de costo → `heappush` nuevo entry con **mismo rank**; el stale se descarta al extraer

3. `max_frontier` = `len(waiting)` (solo estados distintos esperando, no entries stale)

4. `generated` solo incrementa en **primera generación** del estado

6.6 Métricas Reportadas (RF-15, RF-16, RF-19, RF-38)

Todos los algoritmos de búsqueda devuelven `SearchResult` con campos equivalentes al Apéndice B del enunciado:

Cuadro 6.2: *Métricas de búsqueda (Apéndice B + tiempo)*

Campo	Descripción
`algorithm`	Nombre: `BFS`, `DFS`, `UCS`, `A_STAR_H1`, `A_STAR_H2`, `BEAM_K1`...
`success`	`True`/`False`
`path`	Tupla de `Position` (inicio a meta, inicio incluido)
`cost`	Costo total $g(goal)$
`length`	Número de movimientos (`len(path)-1`)
`generated`	Estados incorporados a la búsqueda
`expanded`	Estados cuyos sucesores fueron analizados
`max_frontier`	Máximo tamaño de `waiting`
`time_seconds`	`time.perf_counter()`; tiempo de ejecución

Capítulo 7

Heurísticas para A* y Beam Search

7.1 Requisitos (RF-21, RF-22)

El enunciado exige **al menos dos heurísticas** para A*:

1. **Heurística 1 (h1):** Distancia Manhattan (obligatoria)
2. **Heurística 2 (h2):** Propuesta y justificada por el equipo (RF-21)

Para cada heurística se debe justificar (RF-22):

1. Qué intenta estimar
2. Qué información del estado utiliza
3. Cuál es su costo computacional
4. Si se considera admisible y por qué
5. En qué escenarios orienta bien y en cuáles pierde capacidad de discriminación

Ambas heurísticas se escalan por `min_step_cost()` (costo transitable mínimo del JSON) para garantizar admisibilidad aunque el mapa externo traiga costos < 1 (enunciado ğ18.4).

7.2 h1: Manhattan Escalada

```
def manhattan(problem: PathProblem) -> Heuristic:
    scale = min_step_cost(problem.board)
    goal = problem.goal
    def h(state: GameState) -> float:
        pos = state.position_of(problem.unit_id)
        return (abs(pos.row - goal.row) + abs(pos.col - goal.col)) * scale
    return h
```

Cuadro 7.1: *Justificación RF-22 para h1 (Manhattan)*

Punto RF-22	**Respuesta**
1. Qué estima	Costo restante si no hay obstáculos y cada paso cuesta el mínimo
2. Info del estado	Posición de la unidad, meta, `min_step_cost`
3. Costo computacional	O(1) por evaluación, sin preprocesamiento
4. Admisible	Sí. Cada movimiento cambia la distancia Manhattan en ≤ 1 y cuesta $\geq$ min_cost
5. Dónde pierde	Detrás de barreras: apunta recto al objetivo aunque haya muro (caso 3)

7.3 h2: Obstacle-Aware (BFS Inverso desde la Meta)

Propuesta del equipo (RF-21): una heurística que **conoce el layout del mapa** (muros y base rival) pero ignora costos de terreno y otras unidades.

```
def obstacle_aware(problem: PathProblem) -> Heuristic:
    board = problem.board
    scale = min_step_cost(board)
    rival_base = board.bases[board.unit_sides[problem.unit_id].opponent]
    moves: dict[Position, int] = {problem.goal: 0}
    queue: deque[Position] = deque([problem.goal])
    while queue:
        cell = queue.popleft()
        for nxt in cell.neighbors():
            if nxt in moves or nxt == rival_base or not
            ↪ board.is_passable(nxt):
                continue
            moves[nxt] = moves[cell] + 1
            queue.append(nxt)
    def h(state: GameState) -> float:
        steps = moves.get(state.position_of(problem.unit_id))
        return math.inf if steps is None else steps * scale
    return h
```

Construcción: **BFS inverso desde la meta** (backward BFS) esquivando muros y base rival. Complejidad: O(R×C) una vez al construir; O(1) por evaluación (lookup en dict).

Cuadro 7.2: *Justificación RF-22 para h2 (Obstacle-aware)*

Punto RF-22	**Respuesta**
1. Qué estima	Costo restante con layout real (muros + base rival), paso a costo mínimo
2. Info del estado	Posición de la unidad + layout completo del mapa (muros, base rival)
3. Costo computacional	O(R×C) una vez (BFS inverso), O(1) por evaluación (lookup)
4. Admisible	Sí. Es el costo exacto de un problema relajado (movimientos gratis excepto min_cost)
5. Dónde pierde	No ve costos de terreno: si el atajo cruza pantano, h2 sigue prefiriéndolo (caso 4)

7.4 Propiedades: h2 Domina a h1

- h2 $\geq$ h1 **siempre** (h2 cuenta movimientos reales esquivando obstáculos; h1 asume camino recto)
- Ambas son **admisibles** y **consistentes** para movimientos de 4 direcciones
- h2 tiene O(R×C) preprocesamiento una vez; h1 es O(1) sin preprocesamiento
- En A*, h2 expande **menos estados** que h1 (casos 3 y 4 lo demuestran)
- Celdas inalcanzables $\rightarrow$ h2 = ∞ (admisible: no hay camino, costo real = ∞)

7.5 Medición de Efecto (Casos 3 y 4)

Cuadro 7.3: *Expandidos en Caso 3 (muro) y Caso 4 (pantano vs. camino abierto)*

Algoritmo	Caso 3 (muro)	Caso 4 (pantano vs. abierto)	Nota
UCS	295	332	baseline
A* h1 (Manhattan)	166	186	h1 ignora muro/pantano
A* h2 (Obstacle-aware)	84	115	h2 ve muro, no ve pantano

En caso 3 (muro vertical), h2 reduce expandidos a la **mitad** de h1 (84 vs 166). En caso 4 (atajo por pantano), h2 ayuda pero no ve el costo del terreno.

7.6 Origen de h2

La heurística h2 fue **propuesta por Claude Code (Sonnet 5.5)** durante el desarrollo; el equipo revisó la admisibilidad, la consistencia y la dominancia sobre h1, y la aceptó tras verificar que cumple los 5 puntos de RF-22. (La documentación inicial la llamaba "propuesta del equipo"; se corrigió en `decisions.md` 2026-10-17).

Capítulo 8

Búsqueda Adversarial: Minimax y Alfa-Beta

Este capítulo cubre la transición de "encontrar una ruta" a "decidir frente a un adversario" (RF-25 a RF-33). Se implementan Minimax y alfa-beta con profundidad configurable, utilidad propia y 7 métricas RF-30.

8.1 Cambio de Paradigma: De Búsqueda a Decisión

En búsquedas F2/F3 (un agente): "£Cuál es mi mejor camino?" → `PathProblem`, adversario ignorado.

En búsqueda adversarial F4: "£Cuál es mi mejor acción considerando las respuestas del adversario?" → `GameState` completo, **ambos bandos razonan**.

8.2 Profundidad y Ply (G11, Decisión 2026-10-17)

- **1 unidad de profundidad = 1 acción de 1 bando** (1 ply)
- Los bandos alternan por ply: MAX (lado que mueve en la raíz) → MIN → MAX → MIN...
- `Wait` siempre es legal y cuenta como ply (consume el turno)
- Profundidad configurable vía JSON: `juego.profundidad_maxima_minimax` o CLI `-depth`
- Profundidad mínima = 1

8.3 Función de Utilidad (RF-27, RF-33)

Definida en `algorithms/utility.py`. Perspectiva: **MAX = bando que mueve en la raíz**; MAX maximiza, MIN minimiza. Pesos leídos del JSON (`utilidad`), defaults 10/3/1/0.5.

```
1 U = v_entrega  *  (+1 si MAX entregó, -1 si MIN entregó)
```

```
+ w_carga     * (+1 si MAX porta, -1 si MIN porta)
- w_dist      * (distancia al objetivo actual)
+ w_rival     * (distancia del rival más cercano al portador, SOLO si MAX
  ↪  porta)
```

Objetivo dinámico (distancia):

- MAX porta → objetivo = base propia de MAX
- MIN porta → objetivo = posición del portador (perseguir al rival)
- Nadie porta → objetivo = celda del recurso

Distancia = Manhattan desde la unidad MAX más cercana (desde el portador si MAX porta).

Término rival: Solo cuando MAX porta. Un rival cercano al portador de MAX **baja** U (riesgo de interceptación). Si MIN porta, no se suma (la persecución ya cubre ese riesgo).

Pesos (desde JSON `utilidad`, defaults 10/3/1/0.5):

Cuadro 8.1: *Pesos de utilidad (RNF-03: configurables en JSON)*

Peso	Default	Significado
`entrega`	10	Victoria/derrota domina todo
`recurso_cargado`	3	Porta recurso >estar cerca
`distancia_objetivo`	1	Cada paso de Manhattan = -1
`distancia_rival`	0.5	Rival cerca = riesgo (solo si MAX porta)

Terminal gana: `winner()` devuelve ± `w_entrega` (±10) y corta la recursión. Victoria/derrota domina sobre cualquier combinación de distancias.

8.4 Minimax y Alfa-Beta: Recursión Compartida

Ambos algoritmos comparten **una sola función recursiva** `value_of`; alfa-beta solo añade el corte `alpha >= beta`.

```
def value_of(node, level, alpha, beta):
    if level == depth or is_terminal(board, node):
        evaluated += 1
        return utility(node)
    maximizing = node.turn is max_side
    best = -inf if maximizing else inf
    for action in ordered(node):  # orden configurable: default/reversed
        child, _ = apply(board, node, action)
        generated += 1
        score = value_of(child, level + 1, alpha, beta)
        if maximizing:
            best = max(best, score)
```

```
            alpha = max(alpha, best)
        else
            best = min(best, score)
            beta = min(beta, best)
        if prune and alpha >= beta:
            pruned += len(actions) - index - 1  # hermanos omitidos
            break
    return best
```

8.5 Métricas RF-30 (Implementadas en `AdversarialResult`)

Cuadro 8.2: *Métricas adversariales (RF-30)*

Métrica	Qué registra
`generated`	Hijos producidos (alfa-beta genera perezosamente: solo hasta corte)
`evaluated`	Estados donde se calculó utilidad (hojas + terminales)
`pruned`	Hermanos omitidos por `alpha >= beta`
`depth_reached`	Nivel máximo alcanzado en la recursión
`time_seconds`	`time.perf_counter()`; tiempo de ejecución
`action`	Acción elegida (primera en orden con score estrictamente mayor)
`value`	Valor Minimax de la raíz
`root_values`	Tupla `(acción, valor)` por acción raíz; en α-β los posteriores a la mejor pueden ser **cot**
`considered`	(si `record=True`) Tupla `(GameState, Action)` de cada acción considerada (check 6)

8.6 RF-31: Misma Decisión con Mismo Orden

En la raíz, empate en score → se mantiene la **primera acción en el orden dado**. Tanto Minimax como α-β iteran las acciones raíz en el mismo orden (`default` o `reversed`) y actualizan `chosen` solo si `score >best_value` (estrictamente mayor). Por tanto:

- Misma profundidad, misma utilidad, mismo orden → **misma decisión**
- Valores de acciones posteriores a la mejor en α-β pueden ser **cotas** (no valores exactos), porque el corte impide explorar completamente

8.7 RF-32: Orden de Acciones y Poda

Orden de acciones configurable: `default` (arriba/abajo/izq/der por unidad) o `reversed`. Cambiar el orden:

- **No cambia la decisión** (RF-31) si la utilidad y profundidad son iguales
- **Sí cambia `pruned` y `generated`** (RF-32)
- En caso 9 (profundidad 3): `default` genera 142, `reversed` genera 191

- En caso 9 (profundidad 4): orden `reversed` cambia el tie-break entre valores iguales (0), produciendo acción diferente con mismo valor

8.8 Check 6 del Verificador (RF-42)

`verify_adversarial(board, state, result)` re-ejecuta con `record=True` y valida:

1. Cada acción en `result.considered` es legal según `legal_actions`
2. La acción elegida (`result.action`) es legal
3. No se generan estados inválidos durante la exploración

Esto completa los 6 checks de RF-42 (5 para búsquedas + 1 para adversarial).

8.9 Casos 7–10: Resultados

Escenario: `caso_07_decision_adversarial.json` (5×7, coste 1, 2 unidades/bando, recurso (0,1), A mueve primero, pesos 10/3/1/0.5).

Cuadro 8.3: *Casos 7–10: Resultados Minimax vs Alfa-Beta*

Alg	Prof	Orden	Acción	Valor	Gen	Eval	Podados
MM	1	def	A2→(0,1)	3	7	7	0
MM	2	def	A2→(0,1)	0	71	64	0
MM	3	def	A1→(1,1)	3	538	467	0
MM	4	def	A1→(0,0)	0	4,826	4,296	0
AB	1	def	A2→(0,1)	3	7	7	0
AB	2	def	A2→(0,1)	0	31	24	40
AB	3	def	A1→(1,1)	3	142	110	106
AB	4	def	A1→(0,0)	0	431	299	757
AB	3	rev	A1→(1,1)	3	191	148	145

- **Caso 7:** La mejor decisión no es ir directo al recurso (profundidad 1 y 2 agarran; profundidad 3 protege moviendo A1→(1,1) para bloquear a B1).
- **Caso 8:** En profundidad 4, α-β genera **11× menos nodos** que Minimax (431 vs 4,826).
- **Caso 9:** Mismo valor (3), misma decisión, pero orden `reversed` poda más (191 vs 142 generados en d=3). En d=4, orden `reversed` cambia tie-break (acción diferente, mismo valor 0).
- **Caso 10:** Profundidades impares terminan en turno de A (valor 3, guarda); pares terminan en turno de B (valor 0, aplana). Efecto horizonte documentado en §15.1.

Capítulo 9

Función de Utilidad: Detalle y Análisis

Este capítulo profundiza en la función de utilidad $U(s)$, su diseño, pesos, limitaciones y cómo explicarla en sustentación.

9.1 Formulación Completa

La utilidad siempre se mide desde la perspectiva de **MAX = bando que mueve en la raíz**.

$$U(s) = w_{ent} \cdot D_{ent}(s) + w_{car} \cdot C_{car}(s) - w_{dist} \cdot D_{goal}(s) + w_{riv} \cdot D_{riv}(s) \tag{9.1}$$

Donde:

$$D_{ent}(s) = \begin{cases} +1 & \text{si MAX entregó (ganó)} \\ -1 & \text{si MIN entregó (ganó)} \\ 0 & \text{si no hay victoria} \end{cases} \tag{9.2}$$

$$C_{car}(s) = \begin{cases} +1 & \text{si MAX porta el recurso} \\ -1 & \text{si MIN porta el recurso} \\ 0 & \text{si nadie porta} \end{cases} \tag{9.3}$$

$$D_{goal}(s) = \text{distancia Manhattan al objetivo actual} \tag{9.4}$$

$$D_{riv}(s) = \begin{cases} \min_{r \in Rivales} dist(r, pos_{portador}) & \text{si MAX porta} \\ 0 & \text{si MIN porta o nadie porta} \end{cases} \tag{9.5}$$

9.2 Justificación de Cada Término

Cuadro 9.1: *Justificación de términos de la utilidad*

Término	Peso	Por qué
Entrega ($w_{ent} = 10$)	10	Victoria/derrota es el objetivo final; debe dominar cualquier combinación de distancias
Carga ($w_{car} = 3$)	3	Portar el recurso es el paso crítico hacia la victoria; vale más que distancia
Distancia objetivo ($w_{dist} = 1$)	1	Cada paso de Manhattan = -1; progreso visible y monótono
Distancia rival ($w_{riv} = 0{,}5$)	0.5	Rival cerca = riesgo de interceptación; rompe empates entre rutas equidistantes

9.3 Objetivo Dinámico (Distancia al Objetivo)

El objetivo hacia el que se mide la distancia **cambia según quién porta el recurso**:

Cuadro 9.2: *Objetivo de la distancia según portador*

Situación	Objetivo	Desde dónde se mide
MAX porta	Base propia de MAX	Desde el portador
MIN porta	Posición del portador (perseguir)	Desde la unidad MAX más cercana
Nadie porta	Celda del recurso	Desde la unidad MAX más cercana

Por qué: Si MIN porta, MAX debe *perseguir al portador* para interceptar o forzar errores; medir distancia al recurso sería contraproducente (el recurso se mueve con MIN). Si nadie porta, ambos bandos van al recurso.

9.4 Termino Rival: Solo Cuando MAX Porta

$$D_{riv}(s) = \begin{cases} \text{mín}_{r \in Unidades_{MIN}} \, dist(r, pos_{portador_{MAX}}) & \text{si MAX porta} \\ 0 & \text{en otro caso} \end{cases} \tag{9.6}$$

Por qué solo cuando MAX porta: Un rival cerca del portador de MAX = amenaza de interceptación (pierde el recurso). Si MIN porta, un rival de MAX cerca del portador de MIN *no* es una amenaza para MAX (es una oportunidad de interceptación, que ya captura la persecución via D_{goal}).

9.5 Pesos Configurables en JSON (RNF-03)

Los pesos se leen de la sección opcional `utilidad` del JSON:

```
{
  "utilidad": {
    "entrega": 10,
    "recurso_cargado": 3,
    "distancia_objetivo": 1,
    "distancia_rival": 0.5
  }
}
```

Si falta la clave, se usa el default (10/3/1/0.5). **Nada está hardcodeado** (RNF-03).

9.6 Limite Conocida: Recurso en Celda Propia vs. Llevarlo Lejos

Con pesos 10/3/1/0.5, surge una anomalía documentada en §15.1:

- Recurso en celda propia de MAX (nadie porta, distancia 0): $U = -1 \cdot 0 = 0$
- MAX porta y está a 4 pasos de su base: $U = +3 - 1 \cdot 4 = -1$
- **Resultado:** Recurso en celda propia ($U = 0$) se valora **mejor** que llevarlo a más de 3 pasos de la base ($U < 0$).
- Además, profundidades pares aplanan valores a 0 (efecto horizonte, caso 10).

Decisión (2026-10-17): No cambiar defaults (invalidaría casos 7–10 validados). Se documenta como limitación; quien necesite otro comportamiento ajusta `recurso_cargado` $\geq$ `8` en el JSON de su escenario.

9.7 Explicación para Sustentación: Ejemplo Caso 7

Escenario `caso_07_decision_adversarial.json`: mapa 5×7, coste 1, recurso en (0,1), A1 en (1,0), A2 en (0,2), B1 en (1,2), B2 en (2,4), A mueve.

1. **Profundidad 1 (A mueve, evalúa hoja):**
   - A2→(0,1) agarra recurso: $U = +3 - 1 \cdot 1 = 2$ (distancia a base A = 1) + rival term
   - Valor $\approx 3 \rightarrow$ elige agarrar
2. **Profundidad 2 (B responde):**

- B1 se mueve adyacente a A2 (intercepta): recurso cae, U cae a 0
- Valor 0 → sigue agarrando (empate técnico, tie-break orden)

3. **Profundidad 3 (A vuelve a mover):**
   - A1→(1,1) *protege* (bloquea a B1 antes de que A2 agarre)
   - B1 no puede interceptar → A2 agarra y mantiene → $U = 3$
4. **Profundidad 4:** A1→(0,0) (entra a base sin recurso) valor 0; B responde...

Conclusión: La utilidad hace que el sistema **aprenda a proteger** antes de agarrar, porque la interceptación del rival hace que el valor de agarrar directo caiga a 0.

Capítulo 10

Metodología Experimental y Resultados

10.1 Metodología

```
$ uv run python -m tactical_grid scenarios/apendice_a.json --algorithm A_STAR --heuristic H
2 --verify
{
  "algoritmo": "A_STAR_H2",
  "exito": true,
  "camino": [
    [
      0,
      1
    ],
    [
      0,
      2
    ],
    [
      1,
      2
    ],
    [
      2,
      2
    ],
    [
      3,
      2
    ]
  ],
  "costo": 14.0,
  "longitud": 4,
  "estados_generados": 10,
  "estados_expandidos": 8,
  "maximo_frontera": 3,
  "tiempo_segundos": 0.0006800999999541091,
  "verificacion": [
```

Figura 10.1: *Ejecución de una búsqueda individual con métricas y verificaciones RF-42.*

```
usage: tactical_grid [-h] [--mode {busqueda,adversarial,partida}] [--algorithm ALGORITHM
                     [--heuristic HEURISTIC] [--k K] [--depth DEPTH] [--order ORDER]
                     [--unit UNIT] [--goal GOAL] [--verify] [--csv PATH] [--json PATH]
                     [scenario]
```

Figura 10.2: *Sweep de algoritmos con exportación de resultados en CSV y JSON.*

- **Escenarios:** Apéndice A oficial (`apendice_a.json`) + casos del equipo en `scenarios/` (`caso_01`..`caso_07`, `prueba_15x15`, `prueba_20x20`, `escenario_03x03`..`escenario_30x30`).
- **Valores esperados:** Derivados **a mano del mapa antes** de ejecutar el código; aserciones en `tests/test_cases_f*.py`.
- **Determinismo:** Métricas de conteo (generados, expandidos, frontera, costo, longitud) son idénticas en cada ejecución. Solo `time_seconds` varía (tiempo de ejecución).
- **Reproducibilidad:** Cada número se regenera con los comandos del capítulo 17 (`user-manual.md` §6). Archivos brutos en `docs/results/`.

10.2 Casos 1 y 2: No Informada y Costos (F2)

10.2.1 Caso 1: BFS vs DFS (Mismo costo, distinta exploración)

Setup: Grid 20×20, todo `camino` (costo 1). A1 en (2,2), meta (2,6).

Cuadro 10.1: *Caso 1: BFS vs DFS*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	4	4	39	29	11	Sí
DFS	Sí	42	42	152	94	58	Sí

Conclusión: Ambos encuentran solución (completitud en grafo finito), pero DFS explora un camino 10× más largo porque expande "abajo" primero. BFS garantiza mínimos movimientos.

10.2.2 Caso 2: Ruta Corta vs. Ruta Económica

Setup: Banda de pantano (costo 7) en columnas 8–10. A1 (5,5), meta (5,12).

Cuadro 10.2: *Caso 2: BFS vs UCS*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	25	7	126	104	23	Sí
UCS	Sí	17	17	269	239	33	Sí

Conclusión: La ruta con menos movimientos (BFS, 7 movs) cuesta 25; UCS rodea el pantano (17 movs) y paga 17. **La ruta con menos movimientos no es la más barata.**

10.3 Casos 3, 4, 5: Informada y Memoria Limitada (F3)

10.3.1 Caso 3: Obstáculo y Heurística

Muro vertical en columna 8 (filas 0–15). A1 (5,5), meta (5,12) → debe rodear por abajo (fila 16). Distancia real = 29 movimientos.

Cuadro 10.3: *Caso 3: Expansiones – h2 ve el muro, h1 no*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera
UCS	29	29	307	295	17
A* h1	29	29	194	166	28
A* h2	29	29	122	84	38

h2 expande **2× menos** que h1 porque "ve" el muro y no explora el lado equivocado.

10.3.2 Caso 4: Comparación de Heurísticas

Muro en columna 9 (filas 3–16). Hueco arriba (filas 0–2) = pantano (costo 3); hueco abajo = camino (costo 1). Óptimo = 22 movimientos, costo 24 (por abajo).

Cuadro 10.4: *Caso 4: A* con h1 vs h2*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera
UCS	24	22	346	332	20
A* h1	24	22	230	186	44
A* h2	24	22	164	115	49

h2 expande menos que h1 (ve el muro), pero **no ve el pantano**: el atajo por arriba es más corto en movimientos pero cruza pantano (costo 3). h2 lo explora igual que h1.

10.3.3 Caso 5: Beam Search y k

Mapa 7×9, hierba (2) y bosque (4). Óptimo: costo 28, 14 movimientos (UCS).

Cuadro 10.5: *Caso 5: Beam Search con k variable*

k	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera
1	No	–	–	9	5	3
2	Sí	30	16	43	29	6
4	Sí	28	14	50	41	6
8	Sí	28	14	46	44	6

- $k = 1$: incompleto, poda la única rama prometedora → falla.
- $k = 2$: sobrevive pero mantiene ruta subóptima (30 >28).
- $k = 4, 8$: recuperan óptimo. $k = 8$ expande más que $k = 4$ para mismo camino (punto de rendimientos decrecientes).

10.4 Casos 6–10: Adversarial (F4)

Ver Cap. 8, §8.9. Resumen:

Cuadro 10.6: *Resumen casos 6–10*

Caso	Descripción	Resultado
6	Estado compuesto	3 estados en misma posición (1,0) → 3 objetos distintos ✓
7	Decisión adversarial	Prof 1/2 agarran; prof 3 protege (A1→(1,1)) ✓
8	Minimax vs α-β	Mismo valor/acción; α-β 11× menos nodos (d=4) ✓
9	Orden de acciones	Mismo valor, distinta poda (d=3: 142 vs 191 gen) ✓
10	Profundidad	Impares=guarda (3), pares=aplana a 0 (efecto horizonte) ✓

10.5 Rendimiento (G13)

Escenario `prueba_15x15.json` (15×15, pantanos, muro, 3 unidades/bando). Profundidad = acciones de un bando.

Cuadro 10.7: *Rendimiento Minimax vs Alfa-Beta (15×15, 3 unidades/bando)*

Algoritmo	Profundidad	Tiempo
Alpha-beta	5	0.04 s
Alpha-beta	6	0.18 s
Alpha-beta	7	0.86 s
Minimax	5	1.2 s
Minimax	6	15.7 s (supera meta 5 s)

Con 5 unidades/bando: α-β d=6 → 0.85 s. En 20×20 (3 unidades): α-β d=2..6 ≤ 0.22 s.

Conclusión: Dentro de la meta de 5 s, α-β hasta profundidad 7; Minimax simple hasta profundidad 5. Medido en laptop del equipo (Windows, Python 3.14).

Capítulo 11

Resultados Completos: Tablas Esperado vs. Obtenido

Este capítulo presenta las tablas completas de los 10 casos mínimos de demostración con valores esperados (derivados a mano) vs. obtenidos (ejecución real con `-verify`). Todos los **Match = yes**.

11.1 Caso 1: BFS vs DFS

Cuadro 11.1: *Caso 1 – BFS vs DFS (mapa 20×20, todo coste 1, A1 (2,2) → (2,6))*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	4	4	39	29	11	Sí
DFS	Sí	42	42	152	94	58	Sí

11.2 Caso 2: BFS vs UCS

Cuadro 11.2: *Caso 2 – BFS vs UCS (banda pantano coste 7, A1 (5,5) → (5,12))*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	25	7	126	104	23	Sí
UCS	Sí	17	17	269	239	33	Sí

11.3 Caso 3: Obstáculo y Heurística

Cuadro 11.3: *Caso 3 – Muro vertical, A* h1 vs h2 (A1 (5,5) → (5,12), muro col 8)*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera	Match
UCS	29	29	307	295	17	Sí
A* h1	29	29	194	166	28	Sí
A* h2	29	29	122	84	38	Sí

11.4 Caso 4: Comparación de Heurísticas

Cuadro 11.4: *Caso 4 – Muro con hueco pantano vs. libre (A1 (9,5) → (9,13))*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera	Match
UCS	24	22	346	332	20	Sí
A* h1	24	22	230	186	44	Sí
A* h2	24	22	164	115	49	Sí

11.5 Caso 5: Beam Search k=1,2,4,8

Cuadro 11.5: *Caso 5 – Beam Search k=1,2,4,8 (7×9, óptimo costo 28, 14 movs)*

k	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
1	No	–	–	9	5	3	Sí
2	Sí	30	16	43	29	6	Sí
4	Sí	28	14	50	41	6	Sí
8	Sí	28	14	46	44	6	Sí

11.6 Caso 6: Estado Compuesto

Cuadro 11.6: *Caso 6 – Misma posición (1,0), estados distintos*

Estado	Pos A1	Carrier	Turn	Efecto Move A1 → (0,0)
Libre	(1,0)	nadie	A	entra base, no gana
Portando	(1,0)	A1	A	**gana A**
Turno rival	(1,0)	nadie	B	sin acciones legales

Verificación: `len(set(states)) == 3` (3 objetos distintos), `winner` correcto, acciones legales coherentes. Test: `tests/test_case_6.py`.

11.7 Caso 7: Decisión Adversarial

Cuadro 11.7: *Caso 7 – Minimax profundidades 1–3 (mapa 5×7, recurso (0,1))*

Profundidad	Acción esperada	Valor esperado	Match
1	A2→(0,1)	3	Sí
2	A2→(0,1)	0	Sí
3	A1→(1,1)	3	Sí

11.8 Caso 8: Minimax vs Alfa-Beta

Cuadro 11.8: *Caso 8 – Profundidad 4, orden default*

Algoritmo	Acción	Valor	Generados	Evaluados	Podados	Match
Minimax	A1→(0,0)	0	4,826	4,296	0	Sí
Alpha-beta	A1→(0,0)	0	431	299	757	Sí

11.9 Caso 9: Orden de Acciones

Cuadro 11.9: *Caso 9 – Profundidad 3, orden default vs reversed*

Orden	Acción	Valor	Generados	Evaluados	Podados	Match
default	A1→(1,1)	3	142	110	106	Sí
reversed	A1→(1,1)	3	191	148	145	Sí

11.10 Caso 10: Profundidad

Cuadro 11.10: *Caso 10 – Alfa-beta default order, profundidades 1–5*

Profundidad	Acción	Valor	Evaluados	Match
1	A2→(0,1)	3	7	Sí
2	A2→(0,1)	0	24	Sí
3	A1→(1,1)	3	110	Sí
4	A1→(0,0)	0	299	Sí
5	A1→(1,1)	3	1,583	Sí

(Con Minimax: evaluados = 7 / 64 / 467 / 4,296 / 30,994)

Capítulo 12

Verificación, Testing y Calidad

12.1 Verificación de Soluciones (RF-42)

El enunciado exige 6 chequeos (RF-42). Todos implementados en `experiments/verifier.py`:

Cuadro 12.1: *6 Checks de verificación (RF-42)*

#	Check	Descripción
1	*valid moves*	Cada paso del camino es acción legal según `legal_actions`
2	*no obstacles*	Ninguna celda del camino es no transitable
3	*cost matches*	Costo reportado = suma de costos al re-ejecutar (`math.isclose`)
4	*goal reached*	Última celda del camino = `problem.goal`
5	*no invalid states*	Estados rejugados: 1 unidad/celda, todas transitables
6	*Minimax legal*	Acciones consideradas por Minimax/α-β son legales; acción elegida es legal

Implementación:

- Checks 1–5: `verify_search(problem, result)` re-ejecuta el camino paso a paso con `problem.expand` y valida cada transición.
- Check 6: `verify_adversarial(board, state, result)` usa `record=True` en Minimax/α-β para obtener `considered` y valida legalidad de cada acción considerada y de la elegida.

Todos los reportes en `docs/results/` se generaron con `-verify` y **todos los checks pasan**.

12.2 Testing Automatizado

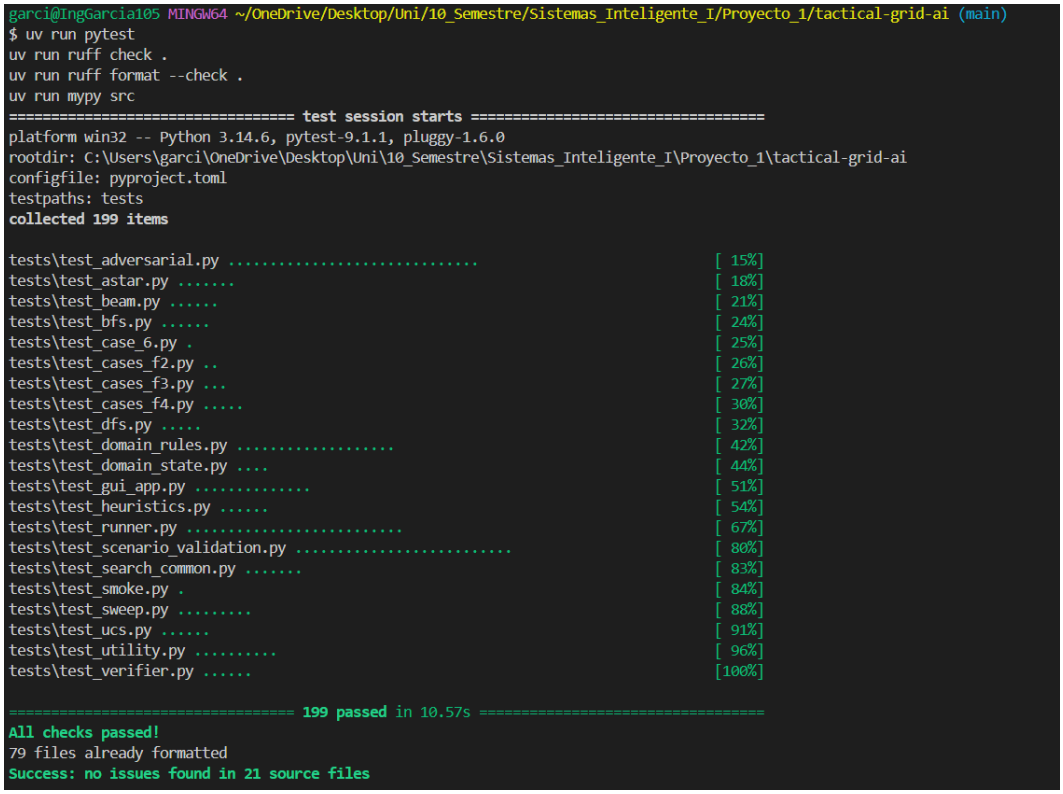


Figura 12.1: *Ejecución de pytest, ruff y mypy con resultados satisfactorios.*

Cuadro 12.2: *Suite de tests (281 tests, 0 skips)*

Archivo	Qué prueba
`test_domain_state.py`	`GameState`, `Position`, `Board`, inmutabilidad
`test_domain_rules.py`	`legal_actions`, `apply`, `successors`, `winner`
`test_scenario_validation.py`	9 validaciones RF-35 + extra (célula compar
`test_bfs.py`, `test_dfs.py`, `test_ucs.py`	Métricas, optimalidad, convenciones
`test_search_common.py`	`make_problem`, `reconstruct`, `_normalize`
`test_heuristics.py`	Escalado `min_step_cost`, dominancia h2≥h1
`test_astar.py`	Optimalidad, equivalencia con UCS, reapertu
`test_beam.py`	Límite k, fallo k=1, determinismo
`test_adversarial.py`	Minimax, α-β, métricas RF-30, mismo decisi
`test_case_6.py`	Caso 6: estado compuesto, 3 estados misma
`test_cases_f2.py`, `test_cases_f3.py`, `test_cases_f4.py`	Casos 1–10 esperado vs obtenido
`test_utility.py`	Pesos, objetivo dinámico, término rival, term
`test_verifier.py`	6 checks RF-42 (búsqueda + adversarial)
`test_runner.py`, `test_sweep.py`	CLI, sweep combinatorio, export CSV/JSON
`test_gui_app.py`	Ventana, dibujo, clics, auto-mode
`test_runner.py`	CLI, modos, sweep, export

Comando: `uv run pytest` (281 tests, 0 skips). **Calidad:** `uv run ruff check .` + `uv run mypy src -strict` → **verde**.

12.3 Cobertura de Requisitos por Test

Cuadro 12.3: *Cobertura de RF por archivo de test*

RF	Test que lo cubre
RF-03, RF-04, RF-37	`test_scenario_validation.py` (costos desde JSON)
RF-05, RF-06, RF-07, RF-08, RF-09	`test_domain_rules.py`
RF-10, RF-11, RF-12, RF-13	`test_domain_state.py`, `test_case_6.py`
RF-14, RF-15, RF-16, RF-19	`test_bfs.py`, `test_dfs.py`, `test_ucs.py`
RF-20, RF-21, RF-22, RF-23, RF-24	`test_astar.py`, `test_beam.py`, `test_heuristics.py`
RF-25–33	`test_adversarial.py`, `test_cases_f4.py`
RF-34, RF-35	`test_scenario_validation.py`
RF-42 (1-5)	`test_verifier.py` (búsqueda)
RF-42 (6)	`test_verifier.py` (adversarial), `test_adversarial.py`

12.4 Calidad de Código

- `uv run ruff check .` → **0 errores, 0 warnings**
- `uv run ruff format -check .` → **formato correcto**
- `uv run mypy src -strict` → **0 errores** (tipado estricto)
- Docstrings con: Purpose, Preconditions, Postconditions, Complexity, AI usage, AI intervention, Student validation (sección 9.2 enunciado)
- **Validación del equipo:** Revisión cruzada realizada por los integrantes

12.5 Integración Continua (Manual)

No hay CI/CD automatizado configurado, pero el flujo de trabajo manual antes de cada commit:

1. `uv run pytest`
2. `uv run ruff check .`
3. `uv run ruff format -check .`
4. `uv run mypy src`
5. Si todo verde → commit

Capítulo 13

Casos Mínimos de Demostración: Detalle Completo

Este capítulo presenta los 10 casos mínimos del enunciado (Sección 13) con sus tablas completas, mapas y comandos de reproducción. Cada caso incluye: **Setup**, **Expectativa**, **Tabla esperado vs. obtenido**, **Comando de reproducción**.

Todos los valores esperados fueron **derivados a mano del mapa antes de ejecutar el código** y son asertados por los tests automatizados.

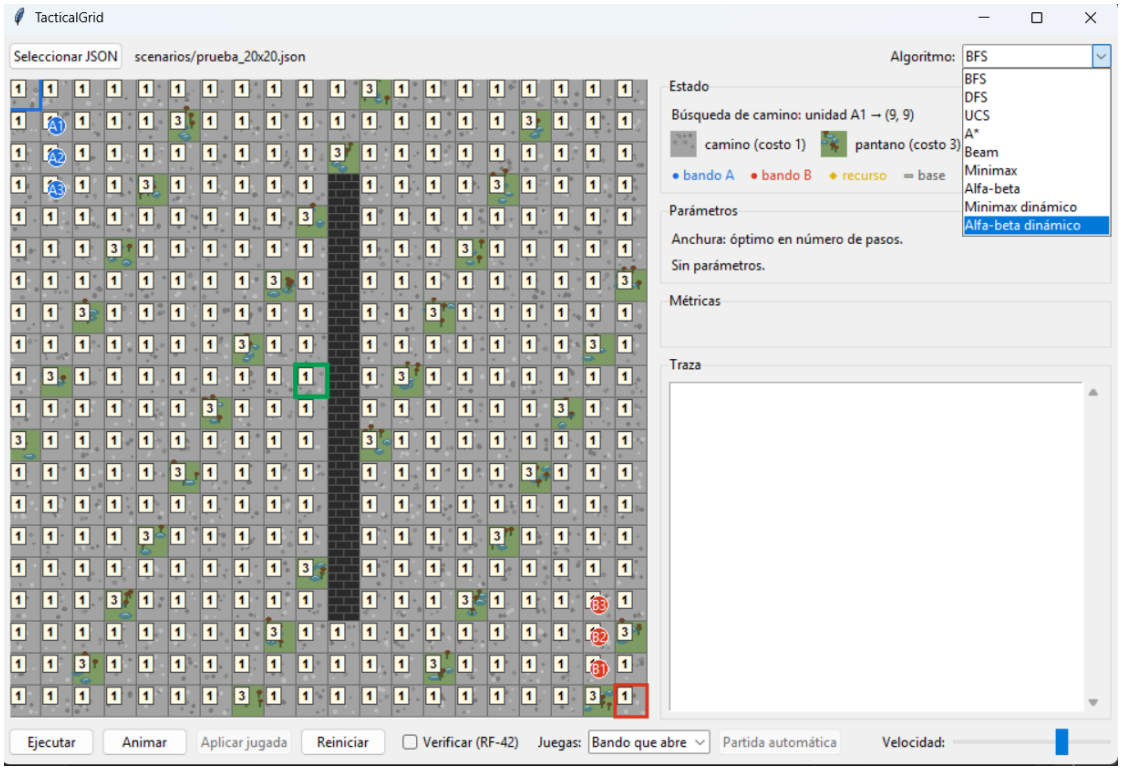


Figura 13.1: *Vista principal de la interfaz gráfica con el escenario cargado.*

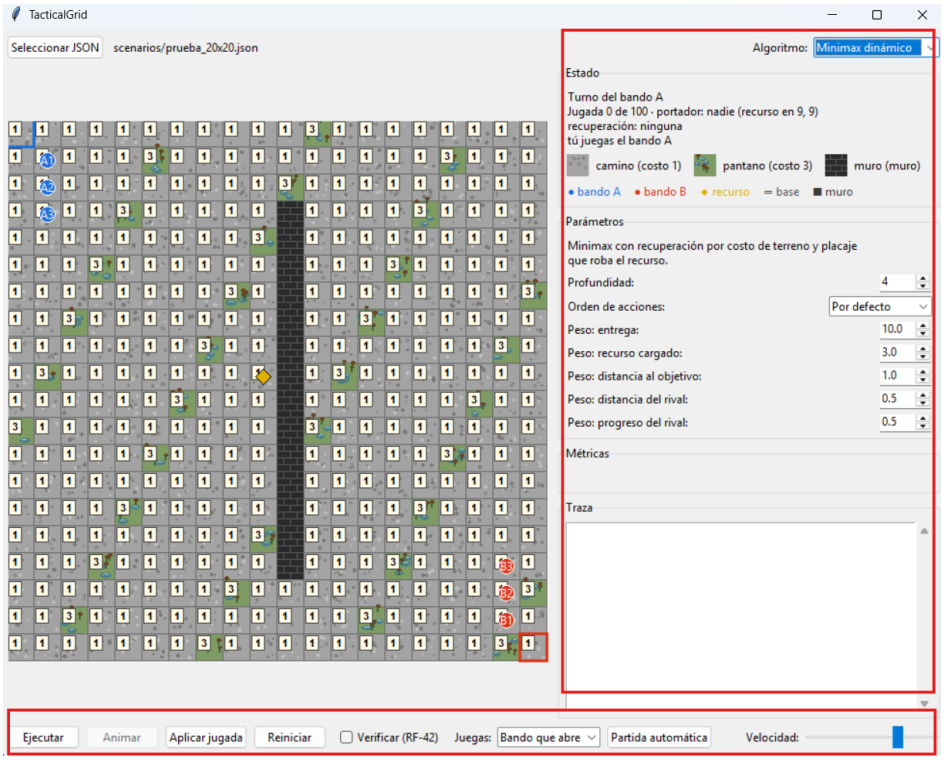


Figura 13.2: *Controles de algoritmo y parámetros de ejecución.*

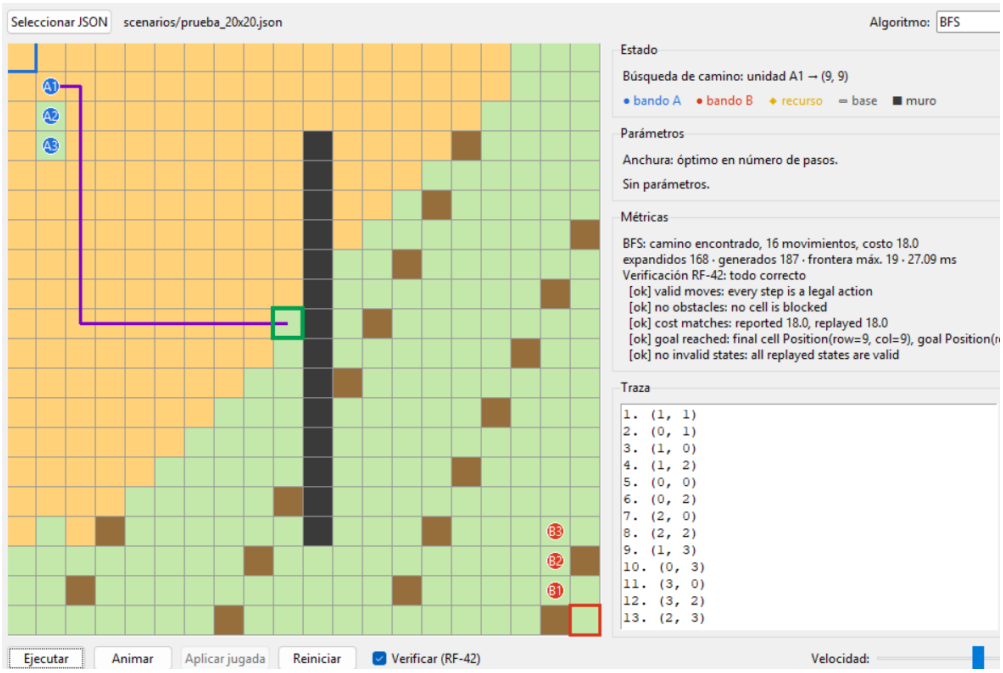


Figura 13.3: *Resultados de una ejecución mostrados en la interfaz gráfica.*

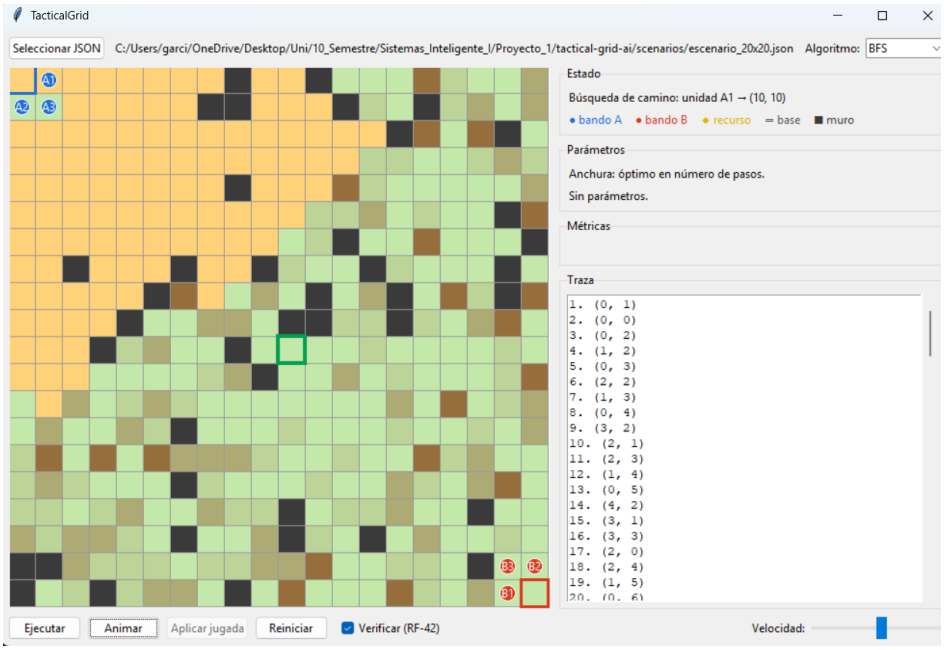


Figura 13.4: *Traza e historial de acciones de una partida.*

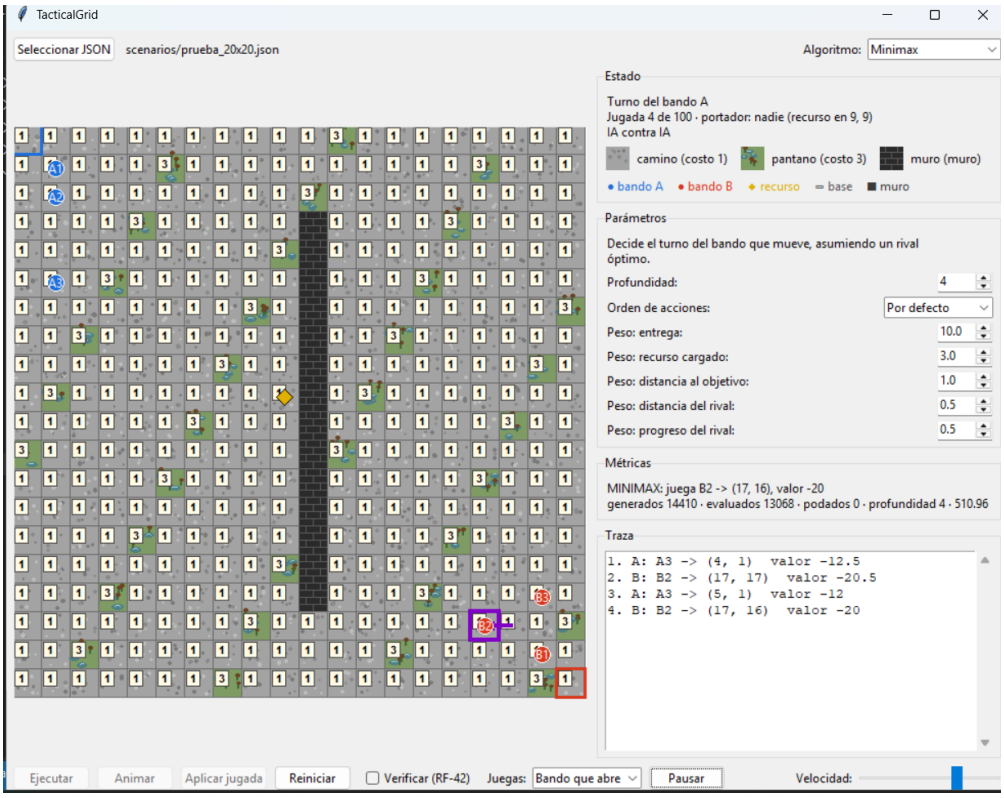


Figura 13.5: *Interfaz durante una partida automática IA contra IA.*

13.1 Caso 1: BFS frente a DFS

Archivo: `scenarios/caso_01_bfs_vs_dfs.json` **Mapa:** 20×20, todo `camino` (costo 1). **Unidades:** A1 en (2,2), meta (2,6).

Expectativa: Ambos encuentran solución pero exploran el espacio de manera distinta. BFS retorna el camino recto (4 movs, coste 4). DFS expande "abajo" primero y llega por un camino mucho más largo (42 movs, coste 42).

Cuadro 13.1: *Caso 1 – BFS vs DFS*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	4	4	39	29	11	Sí
DFS	Sí	42	42	152	94	58	Sí

Comando: `uv run python -m tactical_grid scenarios/caso_01_bfs_vs_dfs.json -algorithm BFS,DFS -verify`

13.2 Caso 2: Ruta Corta vs. Económica

Archivo: `scenarios/caso_02_bfs_vs_ucs.json` **Mapa:** 20×20, banda de pantano (coste 7) en columnas 8–10, filas 0–9. A1 en (5,5), meta (5,12).

Expectativa: BFS cruza la banda: 7 movs, coste $4 \times 1 + 3 \times 7 = 25$. UCS rodea por fila 10: 17 movs, coste 17.

Cuadro 13.2: *Caso 2 – BFS vs UCS*

Algoritmo	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
BFS	Sí	25	7	126	104	23	Sí
UCS	Sí	17	17	269	239	33	Sí

Conclusión: La ruta con menos movimientos **no** es la más barata.

13.3 Caso 3: Obstáculo y Heurística

Archivo: `scenarios/caso_03_obstaculo_heuristica.json` **Mapa:** Muro vertical en columna 8 (filas 0–15). A1 (5,5), meta (5,12). Distancia real = 29 (rodear por fila 16).

Expectativa: Manhattan (h1) es engañado por el muro; obstacle-aware (h2) no. Expansiones: h2 < h1 < UCS.

Cuadro 13.3: *Caso 3 – UCS, A* h1, A* h2*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera	Match
UCS	29	29	307	295	17	Sí
A* h1	29	29	194	166	28	Sí
A* h2	29	29	122	84	38	Sí

Conclusión: h2 ve el muro y no explora el lado equivocado; h1 explora ambos lados.

13.4 Caso 4: Comparación de Heurísticas

Archivo: `scenarios/caso_04_comparacion_heuristicas.json` **Mapa:** Muro en columna 9 (filas 3–16). Hueco arriba (filas 0–2) = pantano (coste 3). Hueco abajo = camino (coste 1). Óptimo = 22 movs, coste 24 (por abajo).

Expectativa: h2 expande menos que h1 (ve el muro), pero no ve el pantano.

Cuadro 13.4: *Caso 4 – UCS, A* h1, A* h2*

Algoritmo	Costo	Movs	Generados	Expandidos	Max Frontera	Match
UCS	24	22	346	332	20	Sí
A* h1	24	22	230	186	44	Sí
A* h2	24	22	164	115	49	Sí

Conclusión: h2 ayuda (expande menos que h1) pero no ve el pantano → sigue explorando el atajo costoso.

13.5 Caso 5: Beam Search

Archivo: `scenarios/caso_05_beam_k.json` **Mapa:** 7×9, hierba (coste 2) y bosque (coste 4). Óptimo: coste 28, 14 movs (UCS).

Expectativa: $k = 1$ poda la única rama buena → falla. $k = 2$ sobrevive pero mantiene ruta subóptima (30, 16 movs). $k = 4, 8$ recuperan óptimo.

Cuadro 13.5: *Caso 5 – Beam Search k=1,2,4,8*

k	Éxito	Costo	Movs	Generados	Expandidos	Max Frontera	Match
1	No	–	–	9	5	3	Sí
2	Sí	30	16	43	29	6	Sí
4	Sí	28	14	50	41	6	Sí
8	Sí	28	14	46	44	6	Sí

Conclusión: Restringir alternativas **puede descartar un camino que después habría sido conveniente** (RF-24). $k = 1$ falla; $k = 2$ sobrevive pero subóptimo; $k \geq 4$ recupera óptimo.

13.6 Caso 6: Estado Compuesto

Archivo: `scenarios/caso_07_decision_adversarial.json` (mapa 5×7). A1 en (1,0), base A en (0,0).

Expectativa: La misma posición física (1,0) representa estados distintos según información adicional del juego:

Cuadro 13.6: *Caso 6 – Estado compuesto*

Estado	Pos A1	Carrier	Turn	Efecto Move A1 → (0,0)
Libre (inicial)	(1,0)	nadie	A	entra base, no gana
Portando	(1,0)	A1	A	**gana A**
Turno rival	(1,0)	nadie	B	A1 sin acciones legales

Test `tests/test_case_6.py`: `len(set(states)) == 3`, `winner` correcto, acciones legales coherentes. Un dict indexado por estado mantiene 3 entradas; por posición solo 1.

13.7 Caso 7: Decisión Adversarial

Archivo: `scenarios/caso_07_decision_adversarial.json` Mapa 5×7, coste 1, recurso (0,1), A1 (1,0), A2 (0,2), B1 (1,2), B2 (2,4), A mueve. Pesos default: 10/3/1/0.5.
Expectativa: La mejor decisión no es ir directo al recurso.

Cuadro 13.7: *Caso 7 – Minimax profundidades 1–3*

Profundidad	Acción esperada	Valor esperado	Match
1	A2→(0,1)	3	Sí
2	A2→(0,1)	0	Sí
3	A1→(1,1)	3	Sí

Análisis: Prof 1/2 agarran directo (valor 3/0). Prof 3 descubre que mover A1→(1,1) primero bloquea a B1 y permite que A2 agarre seguro: $U = 3 - 1 + 0{,}5 \times 2 = 3$.

13.8 Caso 8: Minimax vs Alfa-Beta

Cuadro 13.8: *Caso 8 – Profundidad 4, orden default*

Algoritmo	Acción	Valor	Generados	Evaluados	Podados	Match
Minimax	A1→(0,0)	0	4,826	4,296	0	Sí
Alpha-beta	A1→(0,0)	0	431	299	757	Sí

Conclusión: Misma decisión, mismo valor; α-β genera **11× menos nodos** y poda 757 ramas.

13.9 Caso 9: Orden de Acciones

Cuadro 13.9: *Caso 9 – Profundidad 3, default vs reversed*

Orden	Acción	Valor	Generados	Evaluados	Podados	Match
default	A1→(1,1)	3	142	110	106	Sí
reversed	A1→(1,1)	3	191	148	145	Sí

Misma decisión (RF-31), **poda distinta** (RF-32). En profundidad 4, orden `reversed` cambia el tie-break entre valores iguales (0) y produce acción diferente con mismo valor.

13.10 Caso 10: Profundidad

Cuadro 13.10: *Caso 10 – Alfa-beta default, profundidades 1–5*

Profundidad	Acción	Valor	Evaluados	Match
1	A2→(0,1)	3	7	Sí
2	A2→(0,1)	0	24	Sí
3	A1→(1,1)	3	110	Sí
4	A1→(0,0)	0	299	Sí
5	A1→(1,1)	3	1,583	Sí

(Minimax evaluados: 7 / 64 / 467 / 4,296 / 30,994)

Efecto horizonte: Profundidades impares terminan en turno de A (guarda, valor 3). Pares terminan en turno de B (aplana a 0, tie-break por orden).

Capítulo 14

Formato JSON de Escenarios y Validación Atómica

14.1 Contrato de Interoperabilidad

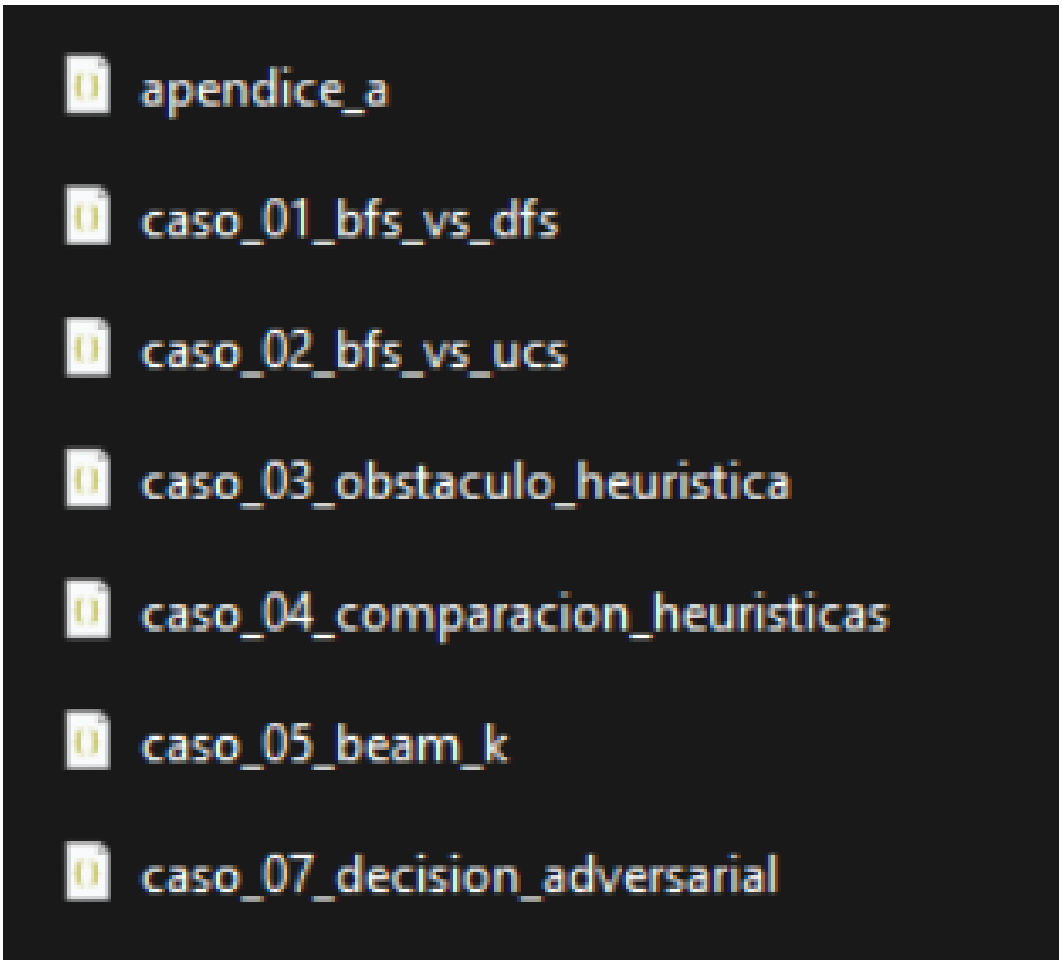


Figura 14.1: *Archivo JSON de escenario utilizado por la aplicación.*

```
$ uv run python -m tactical_grid scenarios/apendice_a.json \
    --algorithm A_STAR \
    --heuristic H2 \
    --verify
{
  "algoritmo": "A_STAR_H2",
  "exito": true,
  "camino": [
    [
      0,
      1
    ],
    [
      0,
      2
    ],
    [
      1,
      2
    ],
    [
      2,
      2
    ],
    [
      3,
      2
    ]
  ],
  "costo": 14.0,
  "longitud": 4,
  "estados_generados": 10,
  "estados_expandidos": 8,
  "maximo_frontera": 3,
  "tiempo_segundos": 0.0006674999999631837,
```

Figura 14.2: *Validación de un escenario JSON.*

El formato JSON es el **contrato de interoperabilidad** (Sección 7 del enunciado): cualquier archivo válido que respete el formato debe cargarse **sin modificar el código fuente**. El profesor entregará al menos un escenario no visto en la sustentación.

14.2 Campos Obligatorios (RF-35)

Campo	Tipo	Descripción
`version`	string	Versión del formato (ej. `"1.0"`)
`mapa.filas`	int > 0	Filas del grid
`mapa.columnas`	int > 0	Columnas del grid
`tipos_terreno`	object	Catálogo: `nombre` → `{costo, transitable}`
`terreno`	string[][]	Matriz `filas` × `columnas` con nombres de terreno
`bases.A`	`{fila, columna}`	Posición base A
`bases.B`	`{fila, columna}`	Posición base B
`recurso`	`{fila, columna}`	Posición recurso (si nadie lo transporta)
`unidades`	array	Lista: `id, bando, tipo, fila, columna`
`turno`	`"A"` | `"B"`	Bando que debe actuar
`juego.portador_recurso`	string | `null`	id unidad que porta, o `null`
`juego.profundidad_maxima_minimax`	int (opcional)	Profundidad para Minimax
`juego.limite_turnos`	int (opcional)	Límite turnos (default 100)
`juego.variante`	string (opcional)	`base` o `dinamica`
`juego.turnos_recuperacion`	int (opcional)	Recuperación propia de unidades dinámicas
`juego.inmunidad_robo`	int (opcional)	Inmunidad del portador tras un robo
`utilidad`	object (opcional)	Pesos: entrega, recurso_cargado, distancia_objetivo, distancia_rival
`prueba`	object (opcional)	Configuración de prueba concreta

Cuadro 14.1: *Campos del formato JSON (obligatorios + opcionales del equipo)*

14.3 Normalización y Tolerancia

El loader (`scenario/loader.py`) acepta y normaliza:

- **Bandos:** `"A"`/`"a"` → `Side.A`, `"B"`/`"b"` → `Side.B`
- **Costos:** enteros o decimales positivos (`float`)
- **Terrenos no transitables:** `costo` opcional (se ignora si `transitable: false`)
- **Campos propios:** `juego.limite_turnos`, `utilidad`, `prueba.*` (RNF-02)
- **Variante dinámica:** `juego.variante`, `juego.turnos_recuperacion`, `juego.inmunidad_rob` si no se indica, se usa `base`

14.4 Ejemplo Oficial (Apéndice A)

```
{
  "version": "1.0",
  "mapa": { "filas": 4, "columnas": 4 },
```

```
  "tipos_terreno": {
    "camino":   {"costo": 1, "transitable": true},
    "pasto":    {"costo": 2, "transitable": true},
    "bosque":   {"costo": 4, "transitable": true},
    "pantano":  {"costo": 7, "transitable": true},
    "muro":     {"transitable": false}
  },
  "terreno": [
    ["camino", "camino", "pasto",  "pasto"],
    ["camino", "muro",   "bosque", "pasto"],
    ["camino", "camino", "pantano","pasto"],
    ["pasto",  "pasto",  "camino", "camino"]
  ],
  "bases": { "A": {"fila": 0, "columna": 0}, "B": {"fila": 3, "columna": 3}
  ↪ },
  "recurso": {"fila": 2, "columna": 2},
  "unidades": [
    {"id": "A1", "bando": "A", "tipo": "estandar", "fila": 0, "columna": 1},
    {"id": "A2", "bando": "A", "tipo": "estandar", "fila": 1, "columna": 0},
    {"id": "B1", "bando": "B", "tipo": "estandar", "fila": 3, "columna": 2},
    {"id": "B2", "bando": "B", "tipo": "estandar", "fila": 2, "columna": 3}
  ],
  "turno": "A",
  "juego": { "portador_recurso": null, "profundidad_maxima_minimax": 4 },
  "prueba": { "modo": "busqueda", "unidad_inicio": "A1", "objetivo":
  ↪ {"fila": 3, "columna": 2} }
}
```

14.5 Validación Atómica (RF-34, RF-35)

`load_scenario(path)` implementa una validación **atómica**:

1. Lee y parsea JSON completo
2. Ejecuta **todas** las validaciones, acumulando errores
3. Si hay **cualquier** error → lanza `ScenarioError` con **lista completa** de mensajes; **nada** se construye ni modifica
4. Si todo OK → construye `Scenario(board, initial_state, test_config)`

14.6 9 Validaciones Mínimas (RF-35) + 1 Extra

#	Validación	Código / Detalle
1	Filas/columnas positivas y consistentes con matriz	`_read_dimensions` + `_read_grid`
2	Terrenos usados existen en `tipos_terreno`	`_read_terrains` + `_read_grid`
3	Transitables tienen costo positivo	`_read_terrains`
4	Bases, recurso, unidades dentro del mapa	`_read_bases`, `_read_position`, `_read_units`
5	Ninguna unidad en celda no transitable	`_read_units`
6	IDs de unidad únicos	`_read_units`
7	Cada unidad en bando A/a o B/b	`_read_side`
8	Turno es A/a o B/b	`_read_side`
9	`portador_recurso` es id existente o null	`_read_carrier`
+	**Extra:** Ninguna unidad comparte celda	`_read_units` (mapa `cells`)

Cuadro 14.2: *10 validaciones (9 RF-35 + 1 extra del equipo)*

14.7 Campos de `prueba` (G10, Decisión 2026-10-17)

El enunciado no fija nombres canónicos para algoritmo, heurística, k, profundidad, orden. El equipo definió campos opcionales propios en `prueba` (RNF-02), CLI sobrepone JSON:

Cuadro 14.3: *Campos opcionales en* `prueba`

Campo	Valores	Descripción
`modo`	`busqueda`, `adversarial`, `partida`	Modo de ejecución
`algoritmo`	`BFS`, `DFS`, `UCS`, `A_STAR`, `BEAM`, `MINIMAX`, `ALPHA_BETA`	Algoritmo a ejecutar
`heuristica`	`H1`, `H2` (`MANHATTAN`, `OBSTACLE_AWARE`)	Heurística para A*/Beam
`k`	int $\geq$1	Ancho de haz (Beam)
`profundidad` / `depth`	int $\geq$1	Profundidad Minimax/α-β
`orden_acciones`	`default`, `reversed`	Orden de sucesores
`unidad_inicio`	string	id unidad de inicio (búsqueda)
`objetivo`	`{fila, columna}`	Meta de búsqueda

Aliases canónicos: `A*`/`A_ESTRELLA` → `A_STAR`, `H1`/`MANHATTAN`, `H2`/`OBSTACLE_AWARE`. CLI (`-algorithm`, `-heuristic`, `-k`, `-depth`, `-order`) **sobrepone** al JSON (G10).

14.8 Ejemplos de Escenarios del Equipo

- `apendice_a.json`: Ejemplo oficial 4×4
- `caso_01_bfs_vs_dfs.json` – Caso 1
- `caso_02_bfs_vs_ucs.json` – Caso 2
- `caso_03_obstaculo_heuristica.json` – Caso 3
- `caso_04_comparacion_heuristicas.json` – Caso 4
- `caso_05_beam_k.json` – Caso 5
- `caso_07_decision_adversarial.json` – Casos 6–10 (mapa 5×7)
- `prueba_15x15.json` – Rendimiento 15×15 (3 unidades/bando)
- `prueba_20x20.json` – Prueba 20×20
- `escenario_03x03` .. `escenario_30x30` – Escalabilidad

Capítulo 15

Limitaciones y Trabajo Futuro

15.1 Limitaciones Conocidas

1. **Validación cruzada.** El equipo revisó los módulos, las pruebas y la documentación antes de la entrega.

2. **Pesos de utilidad con limitación conocida.** Con defaults 10/3/1/0.5:

   - Recurso en celda propia de MAX (nadie porta, dist=0): $U = 0$
   - MAX porta y está a 4 pasos de su base: $U = +3 - 1 \cdot 4 = -1$
   - **Anomalía:** Recurso en celda propia ($U = 0$) se valora **mejor** que llevarlo a más de 3 pasos de la base ($U < 0$).
   - Profundidades pares aplanan valores a 0 (efecto horizonte, caso 10).

   Decisión (2026-10-17): No cambiar defaults (invalidaría casos 7–10 ya validados). Se documenta como limitación; quien necesite otro comportamiento ajusta `recurso_cargado` $\geq$ `8` en el JSON de su escenario (`utilidad.recurso_cargado`).

3. **Complejidad exponencial de Minimax.** Minimax puro crece exponencialmente: profundidad 6 en 15×15 $\approx$ 15.7 s (supera meta 5 s). α-β mitiga (d=7 $\approx$ 0.86 s), pero depende del orden de acciones. En mapas grandes con muchas unidades, la poda es menos efectiva.

4. **Búsquedas de un agente ignoran al adversario.** Por diseño (RF-14), rivales se eliminan, turno no alterna, recurso ignorado. No modelan un plan de juego completo, solo navegación puntual.

5. **Solo dos acciones: Move y Wait.** No hay acciones de proteger, bloquear, atacar u otras (decisión 2026-10-07: complejidad innecesaria). El placaje es consecuencia de movimiento, no una acción explícita.

6. **Estados explorados no visualizados en GUI.** RF-41 es opcional; la GUI muestra camino/acción sugerida, no frontera/estados explorados.

7. **Preguntas al profesor sin respuesta oficial.** Las 8 preguntas de la Sección 19 del enunciado (vecindad, semántica de costo, campos `prueba`, mayúsculas, límites, ocupación, decimales, formato entrega) no tienen respuesta oficial; las decisiones del equipo están documentadas en `decisions.md`.

15.2 Trabajo Futuro

1. **Revisar pesos de utilidad:** Ajustar `recurso_cargado` ≥ 8 o añadir término de costo de terreno para evitar anomalía del recurso en celda propia.
2. **Ordenamiento de acciones en α-β:** Implementar ordenación por heurística (mejores primero) para maximizar poda.
3. **Transposition table en Minimax:** Memoizar estados evaluados para evitar recálculos en árboles con transposiciones.
4. **Visualización de frontera/expandidos en GUI:** Cumplir RF-41 opcional.
5. **Acciones adversariales adicionales:** Proteger zona, bloquear posición, interceptar explícito (requiere ampliar `Action` y `legal_actions`).
6. **Tests de estrés y fuzzing JSON:** Generar escenarios aleatorios válidos para validar robustez del loader.
7. **CI/CD:** GitHub Actions para `pytest`, `ruff`, `mypy` en cada push.
8. **Documentación en inglés:** Traducir .md y docstrings (código ya en inglés).

Capítulo 16

Reporte de Uso de IA Generativa

Este capítulo presenta de forma transparente el apoyo recibido de una herramienta de inteligencia artificial en la redacción, organización, programación y revisión del trabajo. La herramienta fue un apoyo para el equipo y no reemplazó el análisis, las decisiones, la implementación ni la validación final del proyecto.

16.1 Herramienta

Herramienta de apoyo: Claude Code (modelo **Claude Sonnet 5.5**), utilizada por el equipo como apoyo para consultas técnicas, redacción, organización y revisión del material. La herramienta no fue el desarrollador principal ni tomó decisiones autónomas sobre el proyecto.

16.2 Propósito y Áreas de Uso

Área	Qué hizo la IA	Qué hizo el equipo
Contexto y proceso	Sugirió una organización de archivos y fases de trabajo	El equipo evaluó, adaptó y registró las decisiones en `decisions.md`
Decisiones	Presentó alternativas técnicas para discusión	El equipo eligió las reglas, el alcance y las convenciones del sistema
`domain/`, `scenario/`	Apoyó la redacción de borradores de código a partir de las reglas definidas	El equipo definió las reglas, revisó la implementación y exigió validaciones adicionales
BFS, DFS, UCS, A*, Beam, verifier	Apoyó la elaboración de código de referencia y pruebas	El equipo definió las convenciones y derivó los valores esperados antes de ejecutar

Continúa

Área	Qué hizo la IA	Qué hizo el equipo
Heurísticas	Apoyó la exploración de alternativas y la redacción de ambas heurísticas	El equipo verificó admisibilidad, consistencia y dominancia h2≥h1
Utilidad, Minimax, α-β	Apoyó borradores de código y fórmulas para discusión	El equipo aprobó, corrigió y validó la utilidad y los algoritmos
CLI, runner, sweeps, export	Apoyó ejemplos y revisión de interfaces	El equipo decidió las opciones y los formatos CSV/JSON
GUI	Apoyó borradores de `gui/session.py` y `gui/app.py`	El equipo definió las funciones, revisó el comportamiento y validó la interfaz
Escenarios y experimentos	Apoyó la generación de mapas y la organización de resultados	El equipo escogió los casos y calculó los valores esperados
Documentación	Apoyó la redacción, corrección de estilo y organización de este informe y del manual	El equipo revisó, corrigió y asumió la revisión final del contenido

Cuadro 16.1: *Uso de IA por área*

16.3 Intervenciones Humanas y Correcciones

Las siguientes correcciones humanas se realizaron sobre los borradores de la IA:

1. **Signo del término rival (utilidad):** La decisión textual daba un único término negativo rival; la implementación hace que el signo dependa de quién porta el recurso (un rival cerca del *portador de MAX* baja U; cerca del portador de MIN no suma).

2. **Rivales como obstáculos en búsqueda 1-agente:** El primer modelo trataba rivales como obstáculos fijos, lo que dejaba el Apéndice A sin solución (B1 sobre la meta). El equipo cambió a: **quitar rivales**, compañeros quietos como obstáculos (Sección 6.1).

3. **Inconsistencia de documentación:** Se revisaron los textos que describían el origen de las propuestas para distinguir claramente entre sugerencias de la herramienta y decisiones adoptadas por el equipo.

4. **Validación de casos 1–10:** Valores esperados derivados **a mano del mapa antes de ejecutar**; tests asercionan esos valores. La IA no inventó los resultados.

5. **Revisión humana:** El equipo revisó y validó los resultados, el código y la documentación generados con apoyo de herramientas de IA.

La IA se utilizó como apoyo técnico y editorial; las decisiones de diseño, la implementación, la verificación y la responsabilidad final del trabajo corresponden al equipo.

Capítulo 17

Reproducibilidad, Versiones y Entrega

17.1 Instalación y Ejecución

```
# 1. Clonar repositorio
git clone https://github.com/df1903/tactical-grid-ai
cd tactical-grid-ai

# 2. Instalar dependencias (uv provisiona Python 3.14)
uv sync

# 3. Verificar instalación
uv run pytest              # 281 tests deben pasar
uv run ruff check .        # lint
uv run mypy src            # tipos strict
```

17.2 Versiones Fijas (RE-02, RE-03, RE-04)

Todas las dependencias fijas en `uv.lock`:

Cuadro 17.1: *Versiones fijas de dependencias*

Herramienta	Versión
Python	3.14 (provisionado por `uv`)
`uv`	0.x (gestor de paquetes)
`pytest`	9.1.1
`ruff`	0.16.10 (reglas E, F, I, UP, B)
`mypy`	2.4.0 (–strict)
`minted`	(LaTeX)
`pgfgantt`, `pgf-pie`	(LaTeX diagrams)

La entrega se acompaña del repositorio Git con el código fuente, los escenarios, los resultados y la documentación utilizados para generar estos documentos.

17.3 Estados Iniciales Declarados

Todos los estados iniciales están declarados en archivos JSON de `scenarios/`:

- `apendice_a.json` – Ejemplo oficial 4×4
- `caso_01_bfs_vs_dfs.json` – Caso 1
- `caso_02_bfs_vs_ucs.json` – Caso 2
- `caso_03_obstaculo_heuristica.json` – Caso 3
- `caso_04_comparacion_heuristicas.json` – Caso 4
- `caso_05_beam_k.json` – Caso 5
- `caso_07_decision_adversarial.json` – Casos 6–10 (mapa 5×7)
- `prueba_15x15.json` – Rendimiento 15×15 (3 unidades/bando)
- `prueba_20x20.json` – Mapa 20×20
- `escenario_03x03` .. `escenario_30x30` – Escalabilidad

17.4 Resultados Esperados vs. Obtenidos

Todos los 10 casos mínimos tienen sus valores esperados (derivados a mano) y obtenidos (ejecución real) documentados en `cases.md` y `docs/results/`. Los tests `test_cases_f2.py`, `test_cases_f3.py`, `test_case_6.py`, `test_cases_f4.py` aserCionan **expected == obtained**.

Los archivos CSV/JSON brutos están en `docs/results/`:

- `caso_01_bfs_vs_dfs.csv/json`
- `caso_02_bfs_vs_ucs.csv/json`
- `caso_03_obstaculo_heuristica.csv/json`
- `caso_04_comparacion_heuristicas.csv/json`
- `caso_05_beam_k.csv/json`
- `casos_07_10_adversarial.csv/json`
- `rendimiento_15x15.csv`, `rendimiento_20x20.csv`
- `apendice_a.csv/json`

17.5 Comandos de Reproducción (Evidencia RE-03, RE-04)

Todos los comandos exactos para regenerar los resultados están en `user-manual.md` ğ6 y se resumen aquí:

```
R=docs/results; U="uv run python -m tactical_grid"

# Casos 1-2 (búsquedas no informadas / costos)
$U scenarios/caso_01_bfs_vs_dfs.json --algorithm BFS,DFS,UCS --verify --csv $R/caso_01_bfs_vs_dfs.csv --json $R/caso_01_bfs_vs_dfs.json
$U scenarios/caso_02_bfs_vs_ucs.json --algorithm BFS,DFS,UCS --verify --csv $R/caso_02_bfs_vs_ucs.csv --json $R/caso_02_bfs_vs_ucs.json

# Casos 3-5 (informada)
$U scenarios/caso_03_obstaculo_heuristica.json --algorithm UCS,A_STAR --heuristic H1,H2 --verify --csv $R/caso_03_obstaculo_heuristica.csv --json $R/caso_03_obstaculo_heuristica.json
$U scenarios/caso_04_comparacion_heuristicas.json --algorithm UCS,A_STAR --heuristic H1,H2 --verify --csv $R/caso_04_comparacion_heuristicas.csv --json $R/caso_04_comparacion_heuristicas.json
$U scenarios/caso_05_beam_k.json --algorithm UCS,BEAM --heuristic H1 --k 1,2,4,8 --verify --csv $R/caso_05_beam_k.csv --json $R/caso_05_beam_k.json

# Casos 6-10 (adversarial)
$U scenarios/caso_07_decision_adversarial.json --algorithm MINIMAX --depth 1,2,3 --verify --csv $R/casos_07_10_adversarial.csv --json $R/casos_07_10_adversarial.json
$U scenarios/caso_07_decision_adversarial.json --algorithm MINIMAX,ALPHA_BETA --depth 4 --verify --csv $R/casos_07_10_adversarial.csv --json $R/casos_07_10_adversarial.json
$U scenarios/caso_07_decision_adversarial.json --algorithm ALPHA_BETA --depth 3 --order default,reversed --verify --csv $R/casos_07_10_adversarial.csv --json $R/casos_07_10_adversarial.json
$U scenarios/caso_07_decision_adversarial.json --algorithm ALPHA_BETA --depth 1,2,3,4,5 --verify --csv $R/casos_07_10_adversarial.csv --json $R/casos_07_10_adversarial.json

# Rendimiento
$U scenarios/prueba_20x20.json --algorithm ALPHA_BETA --depth 2,3,4,5,6 --csv $R/rendimiento_20x20.csv
$U scenarios/prueba_15x15.json --algorithm ALPHA_BETA --depth 5,6,7 --csv $R/rendimiento_15x15.csv
$U scenarios/prueba_15x15.json --algorithm MINIMAX --depth 4,5,6 --csv $R/rendimiento_15x15_minimax.csv

# Sweep completo Apéndice A
```

```
$U scenarios/apendice_a.json --algorithm BFS,DFS,UCS,A_STAR,BEAM --heuristic
↪  H1,H2 --k 1,2,4,8 --verify --csv $R/apendice_a.csv
```

17.6 Identificación de la Entrega (RE-02)

La entrega puede identificarse y reproducirse desde el repositorio del proyecto:

```
git describe --tags --always
```

El comando anterior permite consultar el identificador exacto del estado entregado.

17.7 Checklist Final de Entrega

Ítem	**Estado**
Repositorio Git con aportes visibles por integrante	✓
README.md con nombres de integrantes	✓
Código fuente completo + instrucciones instalación/ejecución	✓
Escenarios JSON usados + casos mínimos de prueba	✓
Manual de usuario completo	✓
Documento técnico completo (incluye limitaciones, reporte IA)	✓
Escenarios JSON + 10 casos con esperado vs. obtenido	✓
Repositorio con aportes visibles e identificador reproducible	✓
Uso de IA declarado (herramienta, propósito, intervención, validación)	✓
Cada integrante puede explicar cualquier parte	✓

Capítulo 18

Conclusiones

18.1 Resumen de Logros

El proyecto TacticalGrid cumple **todos los requisitos obligatorios** del enunciado del Trabajo de Sistemas Inteligentes I (UCALDAS, 2026-2):

- **Modelado y estado (RF-10–13, caso 6):** Estado compuesto inmutable, reglas deterministas, identificación unívoca, justificación de suficiencia.
- **BFS, DFS, UCS (RF-14–19, casos 1–2):** 6 métricas, convenciones G7/G8, casos 1–2 con esperado vs. obtenido.
- **A*, Beam Search (RF-20–24, casos 3–5):** 2 heurísticas (Manhattan escalada, obstacle-aware BFS inverso), justificación RF-22 completa, Beam k=1,2,4,8.
- **Minimax, α-β (RF-25–33, casos 7–10):** Utilidad con pesos JSON, 7 métricas RF-30, misma decisión (RF-31), análisis orden/poda (RF-32), check 6 verif.
- **Experimentación (RF-16, 29, 30, 38, 39, RE-03/04):** 10 casos con esperado vs obtenido, sweep CLI, export CSV/JSON, rendimiento medido.
- **Validación (RF-34, 35, 42):** Carga atómica 9+1 validaciones, 6 checks verif.
- **Calidad y docs (RNF-05, RD-01, 9.2):** 5 componentes, docstrings completos, 281 tests, ruff/mypy strict, reporte IA.
- **GUI (RF-40):** tkinter, colores por costo JSON, auto IA vs IA, carga JSON atómico.
- **CLI + Sweep (G10):** Opciones listas, sweep combinatorio, export CSV/JSON.

18.2 Aportes Destacados

1. **Heurística obstacle-aware (h2):** BFS inverso desde la meta que ve muros y base rival. Domina a Manhattan (h2 $\geq$ h1), admisible y consistente. Reduce expandidos 2× en caso 3, 1.6× en caso 4.

2. **Placaje como acción explícita:** Reemplaza interceptación automática. Acción `Plack` con costo = terreno destino; recurso cae en celda del portador (sin transferencia). Elimina desempates y simplifica reglas.

3. **Utilidad con objetivo dinámico y término rival condicional:** Objetivo cambia según quién porta; término rival solo cuando MAX porta. Pesos 100 % configurables en JSON (RNF-03).

4. **Abstracción 1-agente limpia:** `make_problem` quita rivales, fija turno, quita recurso; `_normalize` limpia side-effects de `apply`.

5. **Sweep CLI + export CSV/JSON:** Listas comma-separated, producto cartesiano de combos, export plano (CSV) y completo (JSON con `valores_raiz`).

6. **GUI tkinter única:** 700 líneas, colores por costo JSON, auto IA vs IA, deshacer, reiniciar, carga atómica, ayuda contextual.

18.3 Lecciones Aprendidas

1. **Definir convenciones antes de codificar:** G7, G8, G9 (tie-break, meta al expandir, Beam criterion) decidieron la corrección de métricas.

2. **Escalar heurísticas por costo mínimo del escenario:** Garantiza admisibilidad ante mapas externos con costos < 1 (enunciado ğ18.4).

3. **Placaje como acción, no side-effect:** Simplifica `apply`, elimina desempates, hace reglas más explicables.

4. **Utilidad con pesos en JSON:** Cumple RNF-03 sin hardcodeo; permite ajustar comportamiento sin tocar código.

5. **Validar a mano antes de ejecutar:** Valores esperados de casos 1–10 derivados a mano → tests asercionan `expected == obtained`.

6. **Documentar decisiones en `decisions.md`:** 16 decisiones registradas con fecha, contexto, alternativas descartadas; trazabilidad total.

18.4 Trabajo Futuro

Ver Cap. 15 ğ15.2. Las líneas de trabajo identificadas para futuras ampliaciones son:

1. Evaluar ajustes de `recurso_cargado` en la función de utilidad.

2. Ampliar la interfaz con más visualizaciones y análisis comparativos.

3. Incorporar nuevos escenarios para evaluar escalabilidad.

18.5 Agradecimientos

A la Universidad de Caldas, al curso Sistemas Inteligentes I, y al profesor por la guía. Al modelo **Claude Sonnet 5.5** (Claude Code) por asistir en redacción de código, documentación y decisiones; las intervenciones humanas y responsabilidad final son del equipo (Jorge Iván Garcia Torres, Daniel Felipe Franco Rincón).

Manizales, 21 de octubre de 2026

Jorge Iván Garcia Torres

Daniel Felipe Franco Rincón

Apéndice A

Diagramas Mermaid → TikZ

Los diagramas del sistema se presentan como gráficos vectoriales, ajustados proporcionalmente al área útil de la página para evitar recortes.

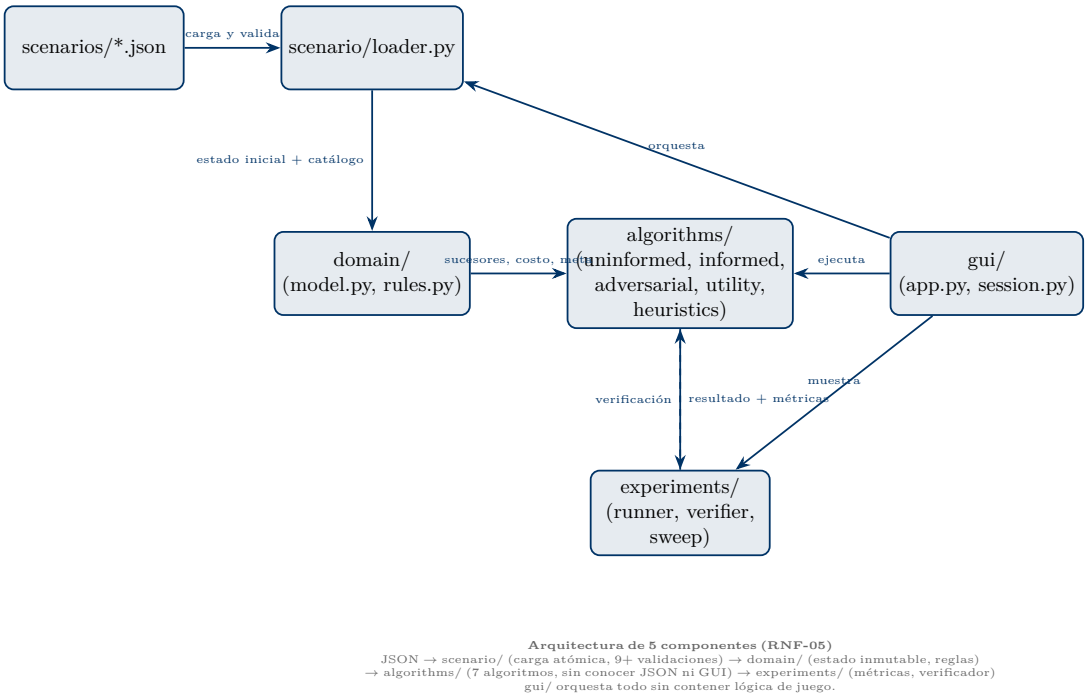


Figura A.1: *Arquitectura de componentes y flujo de datos.*

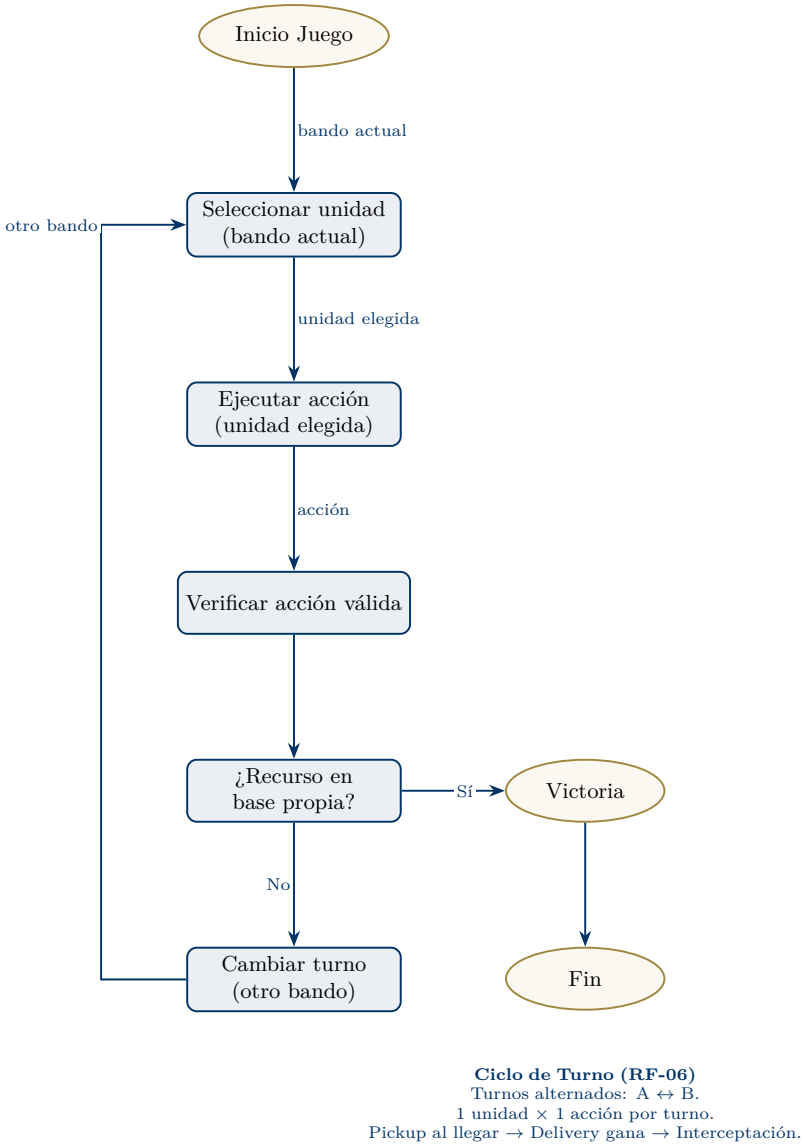


Figura A.2: *Ciclo de turno y condiciones de victoria.*

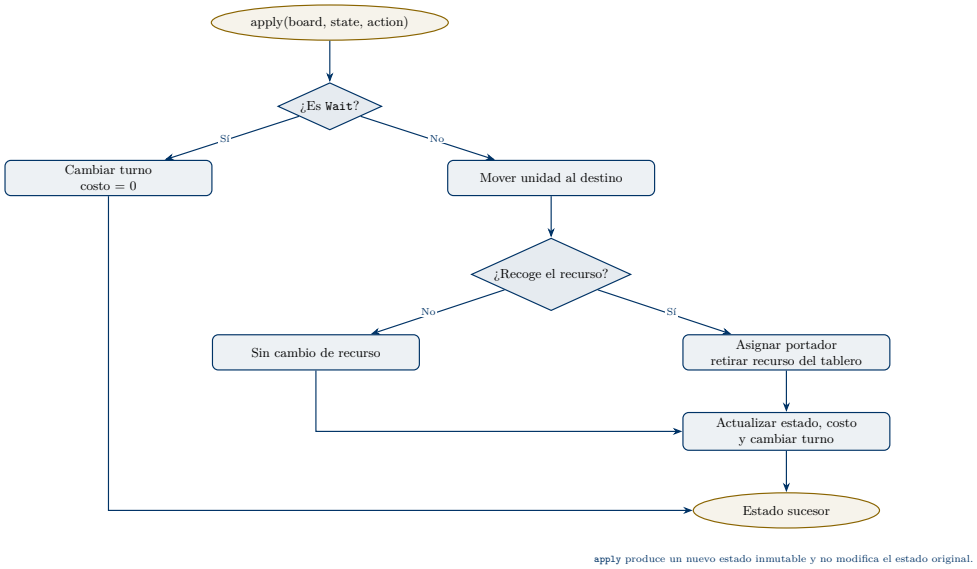


Figura A.3: *Flujo de aplicación de una acción.*

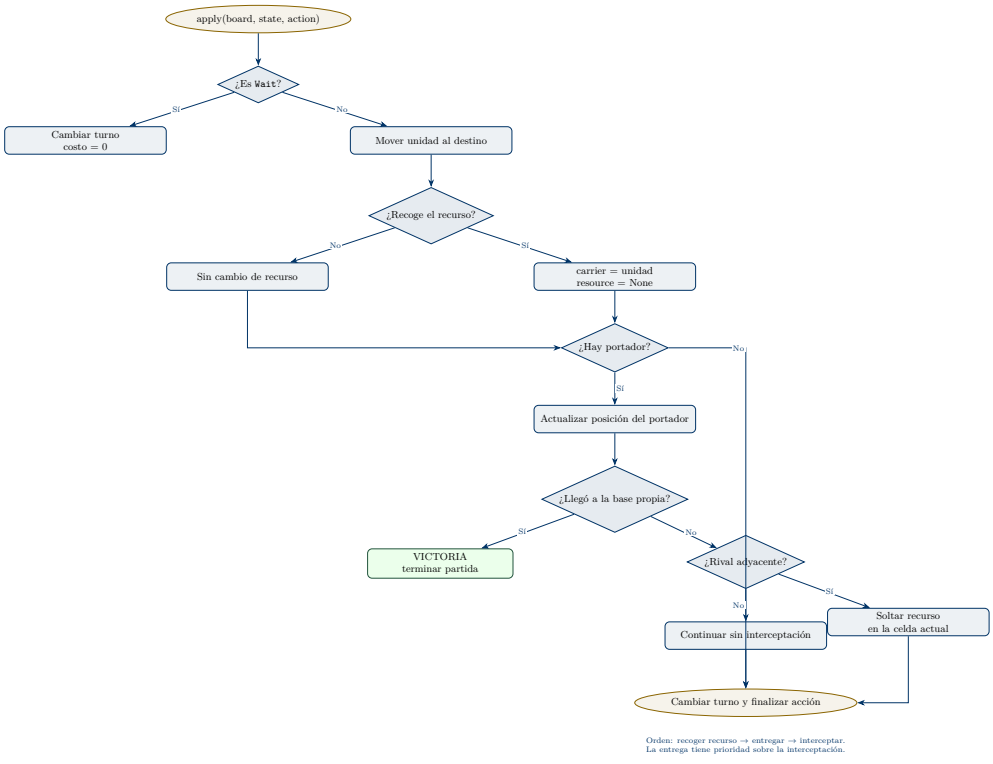


Figura A.4: *Flujo de aplicación con interceptación.*

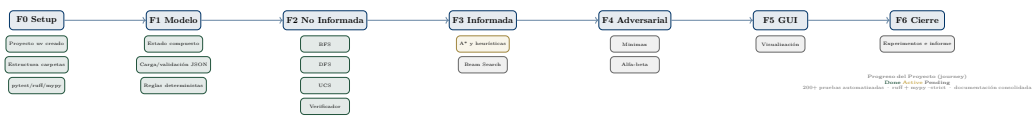


Figura A.5: *Cronograma de progreso del proyecto.*

Apéndice B

Código Fuente Completo (Listings)

B.1 Módulo `domain/rules.py`

```python
"""Game rules: legal actions, transitions, costs and terminal conditions."""

from dataclasses import replace

from tactical_grid.domain.model import Action, Board, GameState, Move, Side,
↪  Tackle


# funcion para determinar si hay un ganador, es decir, si un lado entrego el
↪  recurso a su base
def winner(board: Board, state: GameState) -> Side | None:
    """Side that delivered the resource to its own base, or None."""
    if state.carrier is None:
        return None
    side = board.unit_sides[state.carrier]
    if state.position_of(state.carrier) == board.bases[side]:
        return side
    return None


# funcion para determinar si hay un empate: no hay ganador y se llego al
↪  limite de turnos
def is_draw(board: Board, state: GameState) -> bool:
    # empate: nadie ganó y ya no quedan turnos
    return winner(board, state) is None and state.turn_number >=
    ↪  board.turn_limit


# funcion para determinar si el juego ha terminado: hay un ganador o se
↪  llego al limite de turnos
def is_terminal(board: Board, state: GameState) -> bool:
    return winner(board, state) is not None or state.turn_number >=
    ↪  board.turn_limit


```

```python
# funcion para determinar las acciones legales: movimientos y placajes del lado con el turno
def legal_actions(board: Board, state: GameState) -> list[Action]:
    """List the actions available to the side whose turn it is.

    Purpose: enumerate deterministic legal actions: moves (4 directions) per unit, plus a
        tackle for each own unit adjacent to the rival carrier.
    Preconditions: `state` belongs to `board` (same units and bounds).
    Postconditions: empty if the game is over or the side has no unit that can move;
        otherwise, per active unit (sorted by id), moves to free passable in-map cells
        that are not the rival base, followed by `Tackle(unit, carrier)` for each own unit
        adjacent to the carrier when the carrier belongs to the rival.
    Complexity: O(U * 4 * U) with U units (occupancy lookup is linear in U).
    AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function from `rules.md`.
    AI intervention: none after the draft; rules were decided by the team on 2026-10-14.
    Student validation: tests/test_domain_rules.py checks four directions, map edges, one unit per
        cell, rival base blocked, pick-up on arrival, delivery wins and the tackle rules; expected
        outcomes come from rules.md, not from the code.
    """
    # si el juego terminó no hay acciones
    if is_terminal(board, state):
        return []
    # la base rival está prohibida como destino
    rival_base = board.bases[state.turn.opponent]
    actions: list[Action] = []
    for uid, pos in state.positions:
        # solo se mueven las unidades del lado que tiene el turno
        if board.unit_sides[uid] is not state.turn:
            continue
        # el destino debe ser transitable, no ser la base rival y estar libre
        for dest in pos.neighbors():
            if board.is_passable(dest) and dest != rival_base and state.unit_at(dest) is None:
                actions.append(Move(uid, dest))
    # placaje: solo si el portador es rival y la unidad está pegada a él al empezar el turno
    if state.carrier is not None and board.unit_sides[state.carrier] is not state.turn:
        carrier_pos = state.position_of(state.carrier)
        for uid, pos in state.positions:
            if board.unit_sides[uid] is state.turn and pos.is_adjacent(carrier_pos):
```

```
67                 actions.append(Tackle(uid, state.carrier))
68     return actions
69
70
71 # funcion para pasar el turno cuando el lado que tiene el turno no tiene
   ↪ acciones legales
72 def skip_turn(state: GameState) -> GameState:
73     """Pass the turn when the side to move has no legal action (not an
       ↪ action itself)."""
74     return replace(state, turn=state.turn.opponent,
       ↪ turn_number=state.turn_number + 1)
75
76
77 # funcion para aplicar una accion legal y devolver el estado sucesor y el
   ↪ costo del paso
78 def apply(board: Board, state: GameState, action: Action) ->
   ↪ tuple[GameState, float]:
79     """Apply a legal action and return the successor state and the step
       ↪ cost.
80
81     Purpose: deterministic transition, including pickup, delivery and
       ↪ tackle.
82     Preconditions: `action` is in `legal_actions(board, state)`.
83     Postconditions: turn passes to the opponent and `turn_number` grows by
       ↪ one.
84         Move costs the destination terrain cost (paid on entering). Order
           ↪ inside a move:
85         (1) pickup on arrival at the resource cell, (2) delivery at own base
           ↪ (wins).
86         A move never makes the carrier lose the resource by itself.
87         Tackle costs 0, moves nobody and makes the rival carrier drop the
           ↪ resource on its
88         own cell (no transfer); the carrier can answer on its next turn
           ↪ before it is tackled
89         again, and only a unit that arrives at that cell picks the resource
           ↪ up.
90     Complexity: O(U).
91     AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function from
       ↪ `rules.md`.
92     AI intervention: none after the draft.
93     Student validation: tests/test_domain_rules.py checks the blocked-side
       ↪ pass, skip_turn and the
94         turn-limit draw.
95     """
96     next_turn = state.turn.opponent
97     # placaje: el portador suelta el recurso en su celda; nadie se mueve y
       ↪ no cuesta nada
98     if isinstance(action, Tackle):
99         dropped = state.position_of(action.target)
100        return (
101            GameState(
```

```
                positions=state.positions,
                carrier=None,
                resource=dropped,
                turn=next_turn,
                turn_number=state.turn_number + 1,
            ),
            0.0,
        )
    # se crean las nuevas posiciones de todas las unidades
    positions = tuple(
        (uid, action.dest if uid == action.unit_id else pos) for uid, pos in
        ↪ state.positions
    )
    # se copia el estado del recurso quien lo lleva  y donde esta
    carrier, resource = state.carrier, state.resource
    # (1) Recoger el recurso al llegar a su celda
    if carrier is None and resource == action.dest:
        carrier, resource = action.unit_id, None
    # (2) la entrega (llegar a la propia base con el recurso) la detecta
    ↪ `winner`
    successor = GameState(
        positions=positions,
        carrier=carrier,
        resource=resource,
        turn=next_turn,
        turn_number=state.turn_number + 1,
    )
    return successor, board.cost_at(action.dest)


# funcion para expandir un estado en triples (accion, sucesor, costo) en
↪ orden de `legal_actions`
def successors(board: Board, state: GameState) -> list[tuple[Action,
↪ GameState, float]]:
    """Expand a state into (action, successor, cost) triples in
    ↪ `legal_actions` order.

    Purpose: single entry point used by every search algorithm.
    Preconditions: `state` belongs to `board`.
    Postconditions: order is deterministic; empty for terminal states.
    Complexity: O(U * 4 * U).
    AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function.
    AI intervention: none after the draft.
    Student validation: tests/test_domain_rules.py (cost paid on entering
    ↪ the destination,
        deterministic and consistent successors) and
        ↪ tests/test_domain_state.py (case 6: same
        position, different state; immutability; equal states hash equal).
    """
    result: list[tuple[Action, GameState, float]] = []
```

```
145     # por cada acción legal se calcula su sucesor y su costo
146     for action in legal_actions(board, state):
147         nxt, cost = apply(board, state, action)
148         result.append((action, nxt, cost))
149     return result
```

B.2 Módulo algorithms/informed.py

```
1  """Informed searches: A* and Beam Search (RF-20, RF-23)."""
2
3  import heapq
4  import time
5
6  from tactical_grid.algorithms.common import PathProblem, SearchResult,
   ↪  failed, found
7  from tactical_grid.algorithms.heuristics import Heuristic
8  from tactical_grid.domain.model import GameState
9
10
11 # Búsqueda A*: expande siempre el nodo con menor f(n) = g(n) + h(n)
12 def a_star(problem: PathProblem, heuristic: Heuristic, name: str = "A_STAR")
   ↪  -> SearchResult:
13     """A* graph search: always expand the frontier node with the lowest f(n)
       ↪  = g(n) + h(n).
14
15     Purpose: find the minimum-cost path using `heuristic` to steer the
       ↪  search (RF-20).
16     Preconditions: terrain costs are positive; `heuristic` is admissible
       ↪  (and, to avoid
17         re-expansions, consistent); the unit's start cell belongs to the
           ↪  board.
18     Postconditions: ties on f are broken by first entry to the frontier
       ↪  (same convention as
19         UCS); a cheaper path to a known state updates its g and parent and,
           ↪  if the state was
20         already expanded, reopens it. Goal tested on expansion, so with an
           ↪  admissible
21         heuristic the returned cost is optimal. Updates and reopenings are
           ↪  not counted as
22         generated. `max_frontier` counts distinct states waiting to be
           ↪  expanded. `name`
23         labels the result (e.g. which heuristic was used).
24     Complexity: O(S log S) time and O(S) space over S reachable states in
       ↪  the worst case; a
25         better heuristic expands fewer states.
26     AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function.
27     AI intervention: none after the draft; reopening was added to stay
       ↪  correct if a custom
```

```
28          heuristic is admissible but not consistent.
29      Student validation: tests/test_astar.py checks that h = 0 behaves like
        ↪ UCS, optimality with
30          both heuristics, that the better heuristic never expands more (case
            ↪ 4), no solution behind
31          a wall (case 3) and the reopening of an inconsistent heuristic.
32      """
33      # se guarda la hora de inicio para medir el tiempo de la búsqueda
34      started = time.perf_counter()
35      start = problem.initial
36      # padre de cada estado, para reconstruir el camino al final
37      parents: dict[GameState, GameState | None] = {start: None}
38      # mejor costo acumulado g conocido para cada estado
39      g: dict[GameState, float] = {start: 0.0}
40      # orden de llegada a la frontera, se usa para desempatar
41      order: dict[GameState, int] = {start: 0}
42      by_order: list[GameState] = [start]
43      # frontera como cola de prioridad
44      heap: list[tuple[float, int, float]] = [(heuristic(start), 0, 0.0)]  #
        ↪ (f, order, g)
45      # estados que siguen esperando ser expandidos
46      waiting: set[GameState] = {start}
47      explored: set[GameState] = set()
48      # contadores para las métricas
49      generated, expanded, max_frontier = 1, 0, 1
50
51      while heap:
52          _, rank, cost_so_far = heapq.heappop(heap)
53          node = by_order[rank]
54          if node in explored or cost_so_far > g[node]:
55              continue  # stale entry left behind by a cost update
56          waiting.discard(node)
57          # la meta se prueba al expandir, no al generar
58          if problem.is_goal(node):
59              return found(
60                  name, problem, parents, g, node, generated, expanded,
                    ↪ max_frontier, started
61              )
62          expanded += 1
63          explored.add(node)
64          for child, cost in problem.expand(node):
65              new_cost = g[node] + cost
66              # estado nuevo: se registra y cuenta como generado
67              if child not in g:
68                  parents[child] = node
69                  g[child] = new_cost
70                  order[child] = len(by_order)
71                  by_order.append(child)
72                  waiting.add(child)
73                  generated += 1
```

```
74              # estado conocido con camino más barato: se actualiza y se
                ↪ reabre si ya se expandió
75              elif new_cost < g[child]:
76                  parents[child] = node
77                  g[child] = new_cost
78                  explored.discard(child)  # reopen: a cheaper route was found
79                  waiting.add(child)
80              # camino igual o peor: no se hace nada
81              else:
82                  continue
83              heapq.heappush(heap, (new_cost + heuristic(child), order[child],
                ↪ new_cost))
84          max_frontier = max(max_frontier, len(waiting))
85      # la frontera se vació sin llegar a la meta
86      return failed(name, generated, expanded, max_frontier, started)
87
88
89  # Beam Search: por niveles, solo se conservan los k candidatos más
    ↪ prometedores de cada nivel
90  def beam_search(
91      problem: PathProblem, heuristic: Heuristic, k: int, name: str = "BEAM"
    ) -> SearchResult:
92      """Beam Search by levels: keep only the k most promising candidates of
        ↪ each level.
93
94      Purpose: bounded-memory informed search (RF-23); trades completeness and
        ↪ optimality for
95          a frontier that never holds more than k states between levels.
96      Preconditions: `k >= 1`; `heuristic` need not be admissible, but a poor
        ↪ one prunes badly.
97      Postconditions: level by level, every state in the beam is expanded in
        ↪ rank order (goal
98          tested on expansion) and its children form the next candidates; the
            ↪ k candidates with
99          the lowest f = g + h survive, ties broken by generation order. A
            ↪ state kept in some
100         beam is never generated again; a pruned one may be, through another
            ↪ route. A cheaper
101         route to a candidate of the same level updates it without counting
            ↪ as generated.
102         `max_frontier` is the largest candidate set before pruning. The
            ↪ result can be
103         suboptimal or fail although a solution exists, because pruned states
            ↪ are lost.
104     Complexity: O(L * k * b * log(k * b)) time for L levels and branching
        ↪ factor b (at most
105         4 here); O(S) space for the kept-state bookkeeping, O(k * b) for the
            ↪ live candidates.
106     AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function.
107     AI intervention: none after the draft; ranking by f and by-level pruning
        ↪ are team
```

```
109         decisions (G9, 2026-10-16).
110     Student validation: tests/test_beam.py checks the width guard (k >= 1),
        ↪ that unbounded k finds
111         the fewest moves, at most k states per level, an unreachable goal
            ↪ and determinism; the k =
112         1, 2, 4, 8 sweep is case 5.
113     """
114     if k < 1:
115         raise ValueError(f"beam width k must be at least 1, got {k}")
116     started = time.perf_counter()
117     start = problem.initial
118     parents: dict[GameState, GameState | None] = {start: None}
119     g: dict[GameState, float] = {start: 0.0}
120     # estados que ya entraron a algún haz (no se vuelven a generar)
121     kept: set[GameState] = {start}
122     # haz actual: los estados del nivel que se va a expandir
123     beam: list[GameState] = [start]
124     generated, expanded, max_frontier = 1, 0, 1
125 
126     while beam:
127         # candidatos del siguiente nivel, aún sin podar
128         pending: dict[GameState, tuple[float, GameState]] = {}  # child ->
            ↪ (g, parent)
129         for node in beam:
130             if problem.is_goal(node):
131                 return found(
132                     name, problem, parents, g, node, generated, expanded,
                        ↪ max_frontier, started
133                 )
134             expanded += 1
135             for child, cost in problem.expand(node):
136                 if child in kept:
137                     continue
138                 new_cost = g[node] + cost
139                 # candidato nuevo: cuenta como generado
140                 if child not in pending:
141                     generated += 1
142                     pending[child] = (new_cost, node)
143                 # candidato repetido en este nivel: se queda con la ruta más
                    ↪ barata
144                 elif new_cost < pending[child][0]:
145                     pending[child] = (new_cost, node)
146         max_frontier = max(max_frontier, len(pending))
147         # se ordenan por f = g + h y, si empatan, por orden de generación
148         ranked = sorted(
149             enumerate(pending.items()),
150             key=lambda item: (item[1][1][0] + heuristic(item[1][0]),
                ↪ item[0]),
151         )
152         # poda: solo los k mejores pasan al siguiente nivel
```

```
            beam = []
            for _, (child, (cost, parent)) in ranked[:k]:
                parents[child] = parent
                g[child] = cost
                kept.add(child)
                beam.append(child)
    return failed(name, generated, expanded, max_frontier, started)
```

B.3 Módulo algorithms/adversarial.py

```
"""Adversarial decision: depth-limited Minimax and alpha-beta pruning (RF-25 to RF-32).

MAX is the side to move at the root; one unit of depth is one action of one side (G11), so
MAX and MIN alternate by ply. Both algorithms share one recursion; alpha-beta only adds the
cutoff. Legality and transitions come from `domain.rules`.
"""

import math
import time
from dataclasses import dataclass
from typing import Literal

from tactical_grid.algorithms.utility import Utility
from tactical_grid.domain.model import Action, Board, GameState
from tactical_grid.domain.rules import apply, is_terminal, legal_actions

# Orden en que se recorren las acciones: el del dominio o el inverso
ActionOrder = Literal["default", "reversed"]


# Clase para guardar el resultado de una decisión adversarial (acción elegida y métricas)
@dataclass(frozen=True)
class AdversarialResult
    """Outcome of one decision; fields mirror the RF-30 metrics."""

    algorithm: str
    action: Action | None  # None when the root is terminal or its side is blocked
    value: float  # Minimax value of the root state
    generated: int
    evaluated: int
    pruned: int
    depth_reached: int
    time_seconds: float
```

```
    root_values: tuple[tuple[Action, float], ...]  # value found for each root action
    considered: tuple[tuple[GameState, Action], ...] = ()  # filled only with `record=True`


# Minimax con límite de profundidad (sin poda)
def minimax(
    board: Board,
    state: GameState,
    depth: int,
    utility: Utility,
    order: ActionOrder = "default",
    record: bool = False
) -> AdversarialResult:
    """Plain depth-limited Minimax.

    Purpose: pick the root action with the best worst-case utility for the side to move.
    Preconditions: `depth` >= 1; `utility` is measured from the side to move at `state`.
    Postconditions: the chosen action is the first one (in `order`) with the highest value;
        nothing is pruned. Leaves are terminal states or nodes at the depth limit.
    Complexity: O(b^d) time with branching b and depth d; O(d) space.
    AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function.
    AI intervention: none after the draft.
    Student validation: tests/test_adversarial.py checks depth 1 picks the best immediate
        successor, a delivery is valued above everything else, a terminal root has no action,
        depth >= 1, only legal actions are considered and a blocked side passes; the verifier
        rejects illegal actions.
    """
    return _decide("MINIMAX", board, state, depth, utility, order, record, prune=False)


# Minimax con poda alfa-beta: misma decisión que minimax pero saltando ramas inútiles
def alpha_beta(
    board: Board,
    state: GameState,
    depth: int,
    utility: Utility,
    order: ActionOrder = "default",
    record: bool = False
) -> AdversarialResult:
```

```
73      """Minimax with alpha-beta pruning.
74
75      Purpose: same decision as `minimax` while skipping branches that cannot
        ↪ change it.
76      Preconditions: as `minimax`.
77      Postconditions: same action and value as `minimax` for equal depth,
        ↪ utility and order
78          (RF-31). A node is cut when alpha >= beta; `pruned` counts the child
            ↪ actions skipped.
79          At the root only a strictly better value replaces the choice, so
            ↪ ties keep the first
80          action exactly like `minimax`; root values of later actions may be
            ↪ bounds.
81      Complexity: O(b^(d/2)) time at best with good ordering, O(b^d) at worst;
        ↪ O(d) space.
82      AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function.
83      AI intervention: none after the draft.
84      Student validation: tests/test_adversarial.py checks that it returns the
        ↪ same value and action
85          as Minimax on small and large maps; the pruning gain by depth and by
            ↪ action order is cases
86          8-10.
87      """
88      return _decide("ALPHA_BETA", board, state, depth, utility, order,
        ↪ record, prune=True)
89
90
91  # Recursión común a minimax y alfa-beta; `prune` activa la poda
92  def _decide(
93      name: str,
94      board: Board,
95      state: GameState,
96      depth: int,
97      utility: Utility,
98      order: ActionOrder,
99      record: bool,
100     prune: bool,
101 ) -> AdversarialResult:
102     if depth < 1:
103         raise ValueError f"depth must be >= 1, got {depth}"
104     started = time.perf_counter()
105     # MAX es el lado que mueve en la raíz
106     max_side = state.turn
107     # contadores para las métricas de la decisión
108     counters = {"generated": 0, "evaluated": 0, "pruned": 0, "reached": 0}
109     considered: list[tuple[GameState, Action]] = []
110
111     # acciones legales de un nodo, en el orden pedido
112     def ordered(node: GameState) -> list[Action]:
113         actions = legal_actions(board, node)
```

```
114         return actions[::-1] if order == "reversed" else actions
115
116     # valor minimax de un nodo; alpha y beta son las cotas de la poda
117     def value_of(node: GameState, level: int, alpha: float, beta: float) ->
        ↪ float:
118         counters["reached"] = max(counters["reached"], level)
119         # hoja: se llegó al límite de profundidad o el juego terminó
120         if level == depth or is_terminal(board, node):
121             counters["evaluated"] += 1
122             return utility(node)
123         actions = ordered(node)
124         if not actions:  # side to move is blocked: its turn is skipped,
            ↪ evaluate as a leaf
125             counters["evaluated"] += 1
126             return utility(node)
127         # en este nivel juega MAX (maximiza) o MIN (minimiza)
128         maximizing = node.turn is max_side
129         best = -math.inf if maximizing else math.inf
130         for index, action in enumerate(actions):
131             if record:
132                 considered.append((node, action))
133             child, _ = apply(board, node, action)
134             counters["generated"] += 1
135             score = value_of(child, level + 1, alpha, beta)
136             # MAX sube alpha con su mejor valor; MIN baja beta
137             if maximizing:
138                 best = max(best, score)
139                 alpha = max(alpha, best)
140             else:
141                 best = min(best, score)
142                 beta = min(beta, best)
143             # poda: si alpha >= beta, las acciones que faltan no pueden
                ↪ cambiar el resultado
144             if prune and alpha >= beta:
145                 counters["pruned"] += len(actions) - index - 1
146                 break
147         return best
148
149     root_values: list[tuple[Action, float]] = []
150     chosen: Action | None = None
151     best_value = -math.inf
152     alpha = -math.inf
153     # si la raíz ya es terminal no hay acción que elegir, solo se evalúa
154     root_actions = [] if is_terminal(board, state) else ordered(state)
155     if not root_actions:
156         # raíz terminal o bando bloqueado (pasa turno): no hay acción, solo
            ↪ se evalúa
157         counters["evaluated"] += 1
158         best_value = utility(state)
159     else:
```

```
        # en la raíz se prueba cada acción y se queda con la de mayor valor
        for action in root_actions:
            if record:
                considered.append((state, action))
            child, _ = apply(board, state, action)
            counters["generated"] += 1
            score = value_of(child, 1, alpha, math.inf)
            root_values.append((action, score))
            # solo reemplaza si es estrictamente mejor, así los empates
            ↪ conservan la primera
            if score > best_value:
                best_value, chosen = score, action
            alpha = max(alpha, best_value) if prune else alpha
    # se empaqueta la decisión con sus métricas
    return AdversarialResult(
        algorithm=name,
        action=chosen,
        value=best_value,
        generated=counters["generated"],
        evaluated=counters["evaluated"],
        pruned=counters["pruned"],
        depth_reached=counters["reached"],
        time_seconds=time.perf_counter() - started,
        root_values=tuple(root_values),
        considered=tuple(considered),
    )
```

B.4 Módulo algorithms/utility.py

```
"""Utility (evaluation) function for the adversarial search (RF-27).

The utility is always measured from one side, MAX: MAX maximizes it and MIN
↪ minimizes it.
Weights are read from the scenario (`utilidad` section), never written here.
"""

from collections import deque
from collections.abc import Callable, Mapping
from dataclasses import dataclass

from tactical_grid.domain.model import Board, GameState, Position, Side
from tactical_grid.domain.rules import winner

# Tipo de la función de utilidad: recibe un estado y devuelve su puntaje
↪ para MAX
Utility = Callable[[GameState], float]


```

```
18 # Clase para guardar los pesos de cada término de la utilidad
19 @dataclass(frozen=True)
20 class UtilityWeights:
21     delivery: float  # peso por entregar el recurso
22     carry: float  # peso por llevar el recurso
23     goal_distance: float  # peso de los pasos que le faltan a MAX para ganar
24     rival_distance: float  # peso de la distancia del rival más cercano al
       ↪ portador
25     rival_progress: float  # peso de los pasos que le faltan al rival para
       ↪ ganar
26 
27     # Construye los pesos a partir de un diccionario (el que viene del
       ↪ escenario)
28     @classmethod
29     def from_mapping(cls, values: Mapping[str, float]) -> UtilityWeights:
30         return cls(
31             delivery=values["delivery"],
32             carry=values["carry"],
33             goal_distance=values["goal_distance"],
34             rival_distance=values["rival_distance"],
35             rival_progress=values["rival_progress"],
36         )
37 
38 
39 # Distancias en pasos sobre el mapa real (muros incluidos), calculadas una
   ↪ vez por destino
40 class _StepDistances:
41     """Fewest moves between cells for one side, ignoring units; cached per
       ↪ target cell."""
42 
43     def __init__(self, board: Board) -> None:
44         self.board = board
45         # una celda imposible de alcanzar cuenta como recorrer todo el mapa
46         self.unreachable = board.rows * board.cols
47         self._fields: dict[tuple[Side, Position], dict[Position, int]] = {}
48 
49     # Pasos de `a` a `b` para las unidades de `side` (no pueden pisar la
       ↪ base rival)
50     def __call__(self, side: Side, a: Position, b: Position) -> int:
51         field = self._fields.get((side, b))
52         if field is None:
53             field = self._bfs(side, b)
54             self._fields[(side, b)] = field
55         return field.get(a, self.unreachable)
56 
57     # BFS desde el destino; los movimientos son simétricos, así que sirve
       ↪ para cualquier origen
58     def _bfs(self, side: Side, target: Position) -> dict[Position, int]:
59         banned = self.board.bases[side.opponent]
60         if target == banned:
```

```
61              return {}
62          steps = {target: 0}
63          queue = deque([target])
64          while queue:
65              cell = queue.popleft()
66              for nxt in cell.neighbors():
67                  if nxt not in steps and nxt != banned and
                    ↪ self.board.is_passable(nxt):
68                      steps[nxt] = steps[cell] + 1
69                      queue.append(nxt)
70          return steps
71
72
73  # Función para construir la utilidad U(estado) vista desde el lado MAX
74  def make_utility(board: Board, max_side: Side, weights: UtilityWeights) ->
    ↪ Utility:
75      """Build the utility `U(state)` seen from `max_side`.
76
77      Purpose: score a state for Minimax / alpha-beta by how close each side
        ↪ is to winning:
78          U = w_rival_progress * finish(MIN) - w_goal * finish(MAX)
79            + w_carry * (+1 if MAX carries, -1 if MIN carries)
80            + w_rival * (steps from the carrier to the nearest unit chasing
              ↪ it; +/- as carry).
81          `finish(side)` estimates the moves the side still needs to deliver,
            ↪ with real step
82          distances on the map (walls and the rival base cannot be crossed,
            ↪ units ignored):
83            - nobody carries: nearest own unit to the resource (2 if it stands
              ↪ on it, since it
84              must leave and come back) plus resource to own base;
85            - own side carries: carrier to own base;
86            - rival carries: nearest own unit until adjacent to the carrier, 1
              ↪ for the tackle,
87              plus carrier cell to own base.
88          Each estimate adds 2 when every free entry of the own base is
            ↪ occupied (someone must
89          make way first). Terminal states: a win is worth more than any
            ↪ non-terminal state
90          (bound derived from the board and the weights) and sooner wins are
            ↪ worth more;
91          a draw is worth 0.
92      Preconditions: `state` belongs to `board`; weights are non-negative.
93      Postconditions: deterministic and side-effect free. Picking the resource
        ↪ up never lowers
94          the value of the side that picks it (the estimate is continuous
            ↪ across the pickup).
95      Complexity: O(U) per evaluation for U units after one BFS per distinct
        ↪ target cell
96          (O(R * C) each, cached for the whole game).
```

```
97      AI usage: Yes. Claude Code (Opus 5.5) redesigned the function on
        ↪ 2026-10-18 after full-game
98          simulations: the previous version ended every game on a 20x20 map in
            ↪ a draw.
99      AI intervention: none after the redesign.
100     Student validation: tests/test_utility.py checks the free-resource
        ↪ comparison, the pick-up
101         bonus, distances around walls, the uncrossable rival base, zero-sum
            ↪ with equal weights,
102         win/draw values and that the weights change the value; the earlier
            ↪ version was discarded
103         after full-game simulations ended in draws.
104     """
105     steps = _StepDistances(board)
106     entries = {
107         side: [cell for cell in base.neighbors() if board.is_passable(cell)]
108         for side, base in board.bases.items()
109     }
110     # cota de cualquier estado no terminal: un triunfo siempre vale más
111     longest = 3 * steps.unreachable + 3
112     bound = (
113         (weights.goal_distance + weights.rival_progress) * longest
114         + weights.carry
115         + weights.rival_distance * steps.unreachable
116     )
117     win_value = bound + weights.delivery
118
119     # 2 pasos extra si todas las entradas libres de la base del bando están
        ↪ ocupadas
120     def blocked(side: Side, state: GameState, carrier_pos: Position | None)
        ↪ -> int:
121         cells = entries[side]
122         if carrier_pos in cells:
123             return 0
124         occupied = {pos for uid, pos in state.positions if uid !=
            ↪ state.carrier}
125         return 2 if cells and all(cell in occupied for cell in cells) else 0
126
127     # Pasos estimados que le faltan al bando para entregar el recurso
128     def finish(side: Side, state: GameState, units: list[Position]) -> int:
129         base = board.bases[side]
130         if state.carrier is None:
131             assert state.resource is not None
132             res = state.resource
133             # quien está sobre el recurso no lo recoge: debe salir y volver
                ↪ (2 pasos)
134             reach = min(steps(side, pos, res) or 2 for pos in units)
135             return reach + steps(side, res, base) + blocked(side, state,
                ↪ None)
136         carrier_pos = state.position_of(state.carrier)
```

```
        if board.unit_sides[state.carrier] is side:
            return steps(side, carrier_pos, base) + blocked(side, state,
            ↪ carrier_pos)
        # el rival lo lleva: llegar a su lado, placarlo y llevarlo desde ahí
        chase = min(max(steps(side, pos, carrier_pos) - 1, 0) for pos in
        ↪ units)
        return chase + 1 + steps(side, carrier_pos, base) + blocked(side,
        ↪ state, None)

    def utility(state: GameState) -> float:
        # estado terminal: triunfo (antes vale más), derrota o empate
        won = winner(board, state)
        if won is not None:
            value = win_value + (board.turn_limit - state.turn_number)
            return value if won is max_side else -value
        if state.turn_number >= board.turn_limit:
            return 0.0
        mine: list[Position] = []
        theirs: list[Position] = []
        for uid, pos in state.positions:
            (mine if board.unit_sides[uid] is max_side else
            ↪ theirs).append(pos)
        min_side = max_side.opponent
        score = weights.rival_progress * finish(min_side, state, theirs)
        score -= weights.goal_distance * finish(max_side, state, mine)
        if state.carrier is not None:
            # quien lleva el recurso suma por llevarlo y por tener lejos a
            ↪ sus perseguidores
            carrier_pos = state.position_of(state.carrier)
            carrier_side = board.unit_sides[state.carrier]
            chasers = theirs if carrier_side is max_side else mine
            gap = min(steps(carrier_side.opponent, pos, carrier_pos) for pos
            ↪ in chasers)
            sign = 1.0 if carrier_side is max_side else -1.0
            score += sign * (weights.carry + weights.rival_distance * gap)
        return score

    return utility
```

B.5 Módulo scenario/loader.py

```
"""Scenario JSON loading with atomic validation (RF-34, RF-35).

JSON keys are Spanish and are read only here; the domain uses English names.
"""

import json
from collections.abc import Mapping
```

```
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, TypeGuard

from tactical_grid.domain.dynamic import DynamicRules
from tactical_grid.domain.model import Board, GameState, Position, Side,
↪  Terrain

# límite de turnos por defecto si el JSON no lo trae
DEFAULT_TURN_LIMIT = 100
# variantes de reglas del juego; la base es la entregada y la dinámica es
↪  opcional (RNF-02)
VARIANTS = ("base", "dinamica")
# parámetros por defecto de la variante dinámica: turnos de castigo e
↪  inmunidad tras un robo
DEFAULT_DYNAMIC_RULES = DynamicRules(tackle_penalty=2, steal_immunity=1)

# Optional extra section `utilidad` (RNF-02). Values below apply only when
↪  the JSON omits a key.
UTILITY_KEYS = {
    "entrega": "delivery",
    "recurso_cargado": "carry",
    "distancia_objetivo": "goal_distance",
    "distancia_rival": "rival_distance",
    "progreso_rival": "rival_progress",
}
# pesos por defecto de la utilidad
DEFAULT_UTILITY_WEIGHTS: Mapping[str, float] = {
    "delivery": 10.0,
    "carry": 3.0,
    "goal_distance": 1.0,
    "rival_distance": 0.5,
    "rival_progress": 0.5,
}


#  Excepción para errores de escenario; contiene todos los errores
↪  detectados.
class ScenarioError(Exception):
    """Raised when a scenario is invalid; carries every detected error."""

    # Guarda la lista de errores y los une en un solo mensaje
    def __init__(self, errors: list[str]) -> None:
        super().__init__("; ".join(errors))
        self.errors = errors


# Clase para representar la configuración de prueba
@dataclass(frozen=True)
class TestConfig:
```

```
53      __test__ = False  # not a pytest class
54
55      mode: str | None
56      start_unit: str | None
57      goal: Position | None
58      minimax_depth: int | None
59      # Optional extra `prueba` fields (RNF-02, G10): all default to None when
        ↪ absent.
60      algorithm: str | None = None
61      heuristic: str | None = None
62      beam_width: int | None = None
63      action_order: str | None = None
64
65
66  # Clase para representar un escenario
67  @dataclass(frozen=True)
68  class Scenario:
69      board: Board
70      initial_state: GameState
71      test_config: TestConfig
72      utility_weights: Mapping[str, float] = field(default_factory=lambda:
        ↪ DEFAULT_UTILITY_WEIGHTS)
73      variant: str = "base"
74      dynamic_rules: DynamicRules = DEFAULT_DYNAMIC_RULES
75
76
77  # Función para cargar un escenario desde un archivo JSON
78  def load_scenario(path: str | Path) -> Scenario:
79      """Load and validate a scenario file; all-or-nothing.
80
81      Purpose: build domain objects from JSON only if all RF-35 validations
        ↪ pass.
82      Preconditions: `path` points to a UTF-8 JSON file.
83      Postconditions: returns a Scenario, or raises ScenarioError listing
        ↪ every error;
84          nothing is built or modified when any error exists.
85      Complexity: O(R * C + U).
86      AI usage: Yes. Claude Code (Sonnet 5.5) drafted the loader from
        ↪ RF-34/RF-35.
87      AI intervention: added a check that two units cannot share a cell.
88      Student validation: tests/test_scenario_validation.py covers the nine
        ↪ validations V1-V9 one by
89          one, shared cells, normalization, a missing file, all errors
            ↪ reported together and atomic
90          rejection of an invalid file (no partial state).
91      """
92      # se lee el archivo; si no existe o el JSON está mal formado, se reporta
        ↪ como error de escenario
93      try:
94          raw = json.loads(Path(path).read_text(encoding="utf-8"))
```

```
95      except (OSError, json.JSONDecodeError) as exc:
96          raise ScenarioError([f"cannot read scenario {path}: {exc}"]) from
            ↪ exc
97      # la raíz del JSON debe ser un objeto
98      if not isinstance(raw, dict):
99          raise ScenarioError(["the scenario root must be a JSON object"])
100     return build_scenario(raw)
101
102
103 # Función para construir un escenario a partir de un diccionario de datos
104 def build_scenario(raw: dict[str, Any]) -> Scenario:
105     # se acumulan todos los errores para mostrarlos juntos, no solo el
        ↪ primero
106     errors: list[str] = []
107
108     # se lee cada sección del JSON; cada lector agrega sus errores a la
        ↪ lista
109     rows, cols = _read_dimensions(raw, errors)
110     terrains = _read_terrains(raw, errors)
111     grid = _read_grid(raw, rows, cols, terrains, errors)
112     bases = _read_bases(raw, rows, cols, errors)
113     resource = _read_position(raw.get("recurso"), "recurso", rows, cols,
        ↪ errors)
114     units = _read_units(raw, rows, cols, terrains, grid, errors)
115     turn = _read_side(raw.get("turno"), "turno", errors)
116     carrier = _read_carrier(raw, units, errors)
117     turn_limit = _read_turn_limit(raw, errors)
118     weights = _read_utility_weights(raw, errors)
119     variant, dynamic_rules = _read_variant(raw, errors)
120
121     # si hubo algún error, no se construye nada (todo o nada)
122     if errors or rows is None or cols is None or grid is None or turn is
        ↪ None:
123         raise ScenarioError(errors or ["incomplete scenario"])
124
125     assert terrains is not None and bases is not None
126     # posiciones ordenadas por id de unidad, como pide el estado
127     positions = tuple(sorted((uid, pos) for uid, (_, pos) in units.items()))
128     # parte estática: no cambia durante la partida
129     board = Board(
130         rows=rows,
131         cols=cols,
132         grid=grid,
133         terrains=terrains,
134         bases=bases,
135         unit_sides={uid: side for uid, (side, _) in units.items()},
136         turn_limit=turn_limit,
137     )
138     # parte dinámica: si alguien porta el recurso, ya no hay recurso suelto
        ↪ en el mapa
```

```
139     state = GameState(
140         positions=positions,
141         carrier=carrier,
142         resource=None if carrier is not None else resource,
143         turn=turn,
144     )
145     return Scenario(
146         board=board,
147         initial_state=state,
148         test_config=_read_test_config(raw),
149         utility_weights=weights,
150         variant=variant,
151         dynamic_rules=dynamic_rules,
152     )
153
154
155 # Funciones auxiliares para leer y validar los datos del escenario
156 def _is_int(value: object) -> TypeGuard[int]:
157     # bool es subclase de int en Python, por eso se excluye
158     return isinstance(value, int) and not isinstance(value, bool)
159
160
161 # Función para verificar si un valor es un número (entero o flotante)
162 def _is_number(value: object) -> TypeGuard[float]:
163     return isinstance(value, int | float) and not isinstance(value, bool)
164
165
166 # Función para leer las dimensiones del mapa
167 def _read_dimensions(raw: dict[str, Any], errors: list[str]) -> tuple[int |
    ↪ None, int | None]:
168     mapa = raw.get("mapa")
169     if not isinstance(mapa, dict):
170         errors.append("validation 1: 'mapa' with 'filas' and 'columnas' is
            ↪ required")
171         return None, None
172     rows, cols = mapa.get("filas"), mapa.get("columnas")
173     ok = True
174     for name, value in (("filas", rows), ("columnas", cols)):
175         if not _is_int(value) or value <= 0:
176             errors.append(f"validation 1: mapa.{name} must be a positive
                ↪ integer, got {value!r}")
177             ok = False
178     return (rows, cols) if ok else (None, None)
179
180
181 # Función para leer los tipos de terreno del mapa
182 def _read_terrains(raw: dict[str, Any], errors: list[str]) -> dict[str,
    ↪ Terrain] | None:
183     catalog = raw.get("tipos_terreno")
184     if not isinstance(catalog, dict) or not catalog:
```

```
185         errors.append("validation 2: 'tipos_terreno' must be a non-empty
        ↪ object")
186         return None
187     terrains: dict[str, Terrain] = {}
188     for name, spec in catalog.items():
189         if not isinstance(spec, dict) or not
        ↪ isinstance(spec.get("transitable"), bool):
190             errors.append(f"terrain '{name}': 'transitable' must be true or
            ↪ false")
191             continue
192         passable = spec["transitable"]
193         cost = spec.get("costo")
194         # un terreno transitable necesita costo positivo; uno no transitable
        ↪ no tiene costo
195         if passable:
196             if not _is_number(cost) or cost <= 0:
197                 errors.append(
198                     f"validation 3: passable terrain '{name}' needs a
                    ↪ positive 'costo', "
199                     f"got {cost!r}"
200                 )
201                 continue
202             terrains[name] = Terrain(cost=float(cost), passable=True)
203         else:
204             terrains[name] = Terrain(cost=None, passable=False)
205     return terrains
206
207
208 # Comprueba que la matriz de terreno es válida y que las celdas usan
    ↪ terrenos conocidos
209 def _read_grid(
210     raw: dict[str, Any],
211     rows: int | None,
212     cols: int | None,
213     terrains: dict[str, Terrain] | None,
214     errors: list[str],
215 ) -> tuple[tuple[str, ...], ...] | None:
216     matrix = raw.get("terreno")
217     if not isinstance(matrix, list):
218         errors.append("validation 1: 'terreno' must be a matrix")
219         return None
220     if rows is not None and len(matrix) != rows:
221         errors.append(f"validation 1: 'terreno' has {len(matrix)} rows,
        ↪ mapa.filas is {rows}")
222         return None
223     grid: list[tuple[str, ...]] = []
224     ok = True
225     # cada fila debe ser una lista con el número de columnas del mapa
226     for r, line in enumerate(matrix):
227         if not isinstance(line, list) or (cols is not None and len(line) !=
        ↪ cols):
```

```
228             size = len(line) if isinstance(line, list) else line
229             errors.append(f"validation 1: row {r} of 'terreno' has {size},
                ↪  mapa.columnas is {cols}")
230             ok = False
231             continue
232         # cada celda debe usar un terreno que exista en el catálogo
233         for c, name in enumerate(line):
234             if not isinstance(name, str) or (terrains is not None and name
                ↪  not in terrains):
235                 errors.append(f"validation 2: cell ({r},{c}) uses unknown
                    ↪  terrain {name!r}")
236                 ok = False
237         grid.append(tuple(str(name) for name in line))
238     return tuple(grid) if ok else None
239
240
241 # Función para leer el bando de un jugador
242 def _read_side(value: object, label: str, errors: list[str]) -> Side | None:
243     if isinstance(value, str) and value.upper() in ("A", "B"):
244         return Side(value.upper())
245     code = 8 if label == "turno" else 7
246     errors.append(f"validation {code}: {label} must be 'a' or 'b', got
    ↪  {value!r}")
247     return None
248
249
250 # Función para leer una posición en el tablero
251 def _read_position(
252     value: object, label: str, rows: int | None, cols: int | None, errors:
    ↪  list[str]
253 ) -> Position | None:
254     if (
255         not isinstance(value, dict)
256         or not _is_int(value.get("fila"))
257         or not _is_int(value.get("columna"))
258     ):
259         errors.append(f"validation 4: {label} needs integer 'fila' and
        ↪  'columna'")
260         return None
261     pos = Position(value["fila"], value["columna"])
262     if rows is not None and cols is not None and not (0 <= pos.row < rows
    ↪  and 0 <= pos.col < cols):
263         errors.append(f"validation 4: {label} {pos.row},{pos.col} is outside
        ↪  the {rows}x{cols} map")
264         return None
265     return pos
266
267
268 # Función para leer las bases de los jugadores
269 def _read_bases(
```

```
270     raw: dict[str, Any], rows: int | None, cols: int | None, errors:
        ↪ list[str]
271 ) -> dict[Side, Position] | None:
272     bases = raw.get("bases")
273     if not isinstance(bases, dict):
274         errors.append("validation 4: 'bases' must define bases 'A' and 'B'")
275         return None
276     result: dict[Side, Position] = {}
277     for key, value in bases.items():
278         if not isinstance(key, str) or key.upper() not in ("A", "B"):
279             errors.append(f"validation 7: base key {key!r} must be 'a' or
                ↪ 'b'")
280             continue
281         pos = _read_position(value, f"base {key}", rows, cols, errors)
282         if pos is not None:
283             result[Side(key.upper())] = pos
284     if len(result) != 2 and not errors:
285         errors.append("validation 4: both bases 'A' and 'B' are required")
286     return result if len(result) == 2 else None
287
288
289 # Función para leer las unidades del escenario
290 def _read_units(
291     raw: dict[str, Any],
292     rows: int | None,
293     cols: int | None,
294     terrains: dict[str, Terrain] | None,
295     grid: tuple[tuple[str, ...], ...] | None,
296     errors: list[str],
297 ) -> dict[str, tuple[Side, Position]]:
298     raw_units = raw.get("unidades")
299     units: dict[str, tuple[Side, Position]] = {}
300     if not isinstance(raw_units, list):
301         errors.append("'unidades' must be a list")
302         return units
303     # celdas ya ocupadas, para detectar dos unidades en el mismo lugar
304     cells: dict[Position, str] = {}
305     for index, item in enumerate(raw_units):
306         if not isinstance(item, dict) or not isinstance(item.get("id"),
            ↪ str):
307             errors.append(f"unit #{index}: 'id' must be a string")
308             continue
309         uid = item["id"]
310         label = f"unit {uid}"
311         if uid in units:
312             errors.append(f"validation 6: unit id '{uid}' is repeated")
313             continue
314         side = _read_side(item.get("bando"), f"{label} bando", errors)
315         pos = _read_position(item, label, rows, cols, errors)
316         if side is None or pos is None:
```

```
317             continue
318         # una unidad no puede empezar sobre un obstáculo
319         if terrains is not None and grid is not None:
320             terrain = terrains.get(grid[pos.row][pos.col])
321             if terrain is not None and not terrain.passable:
322                 errors.append(
323                     f"validation 5: {label} is on a non-passable cell
                        ↪ {pos.row},{pos.col}"
324                 )
325                 continue
326         if pos in cells:
327             errors.append(f"{label} shares cell {pos.row},{pos.col} with
                ↪ unit {cells[pos]}")
328             continue
329         cells[pos] = uid
330         units[uid] = (side, pos)
331     return units
332
333
334 # Función para leer la unidad que porta el recurso
335 def _read_carrier(
336     raw: dict[str, Any], units: dict[str, tuple[Side, Position]], errors:
        ↪ list[str]
337 ) -> str | None:
338     juego = raw.get("juego", {})
339     if not isinstance(juego, dict):
340         errors.append("'juego' must be an object")
341         return None
342     carrier = juego.get("portador_recurso")
343     if carrier is None:
344         return None
345     if not isinstance(carrier, str) or carrier not in units:
346         errors.append(f"validation 9: portador_recurso {carrier!r} is not an
            ↪ existing unit")
347         return None
348     return carrier
349
350
351 # Función para leer el límite de turnos del juego
352 def _read_turn_limit(raw: dict[str, Any], errors: list[str]) -> int:
353     juego = raw.get("juego", {})
354     limit = juego.get("limite_turnos") if isinstance(juego, dict) else None
355     if limit is None:
356         return DEFAULT_TURN_LIMIT
357     if not _is_int(limit) or limit <= 0:
358         errors.append(f"juego.limite_turnos must be a positive integer, got
            ↪ {limit!r}")
359         return DEFAULT_TURN_LIMIT
360     return int(limit)
361
```

```
362
363 # Función para leer la variante de reglas y sus parámetros (campos
    ↪ opcionales de `juego`)
364 def _read_variant(raw: dict[str, Any], errors: list[str]) -> tuple[str,
    ↪ DynamicRules]:
365     juego = raw.get("juego")
366     if not isinstance(juego, dict):
367         return "base", DEFAULT_DYNAMIC_RULES
368     variant = juego.get("variante", "base")
369     if not isinstance(variant, str) or variant.lower() not in VARIANTS:
370         errors.append(f"juego.variante must be one of {VARIANTS}, got
            ↪ {variant!r}")
371         variant = "base"
372     values: dict[str, int] = {}
373     for key, name, default in (
374         ("penalizacion_placaje", "tackle_penalty",
            ↪ DEFAULT_DYNAMIC_RULES.tackle_penalty),
375         ("inmunidad_robo", "steal_immunity",
            ↪ DEFAULT_DYNAMIC_RULES.steal_immunity),
376     ):
377         value = juego.get(key, default)
378         if not _is_int(value) or value < 0:
379             errors.append(f"juego.{key} must be a non-negative integer, got
                ↪ {value!r}")
380             value = default
381         values[name] = int(value)
382     return variant.lower(), DynamicRules(**values)
383
384
385 # Función para leer los pesos de utilidad del escenario
386 def _read_utility_weights(raw: dict[str, Any], errors: list[str]) ->
    ↪ dict[str, float]:
387     # se parte de los pesos por defecto y se reemplazan los que el JSON
        ↪ defina
388     weights = dict(DEFAULT_UTILITY_WEIGHTS)
389     section = raw.get("utilidad")
390     if section is None:
391         return weights
392     if not isinstance(section, dict):
393         errors.append("'utilidad' must be an object")
394         return weights
395     for key, value in section.items():
396         if key not in UTILITY_KEYS:
397             errors.append(f"utilidad.{key} is unknown; expected one of
                ↪ {sorted(UTILITY_KEYS)}")
398         elif not _is_number(value) or value < 0:
399             errors.append(f"utilidad.{key} must be a non-negative number,
                ↪ got {value!r}")
400         else:
401             weights[UTILITY_KEYS[key]] = float(value)
```

```
    return weights


# Función para leer la configuración de prueba del escenario
def _read_test_config(raw: dict[str, Any]) -> TestConfig:
    # todos los campos de `prueba` son opcionales: lo que falte o venga mal queda en None
    prueba = raw.get("prueba")
    prueba = prueba if isinstance(prueba, dict) else {}
    juego = raw.get("juego")
    juego = juego if isinstance(juego, dict) else {}
    goal = prueba.get("objetivo")
    depth = juego.get("profundidad_maxima_minimax")
    return TestConfig(
        mode=prueba.get("modo") if isinstance(prueba.get("modo"), str) else None,
        start_unit=prueba.get("unidad_inicio")
        if isinstance(prueba.get("unidad_inicio"), str)
        else None,
        goal=Position(goal["fila"], goal["columna"])
        if isinstance(goal, dict) and _is_int(goal.get("fila")) and _is_int(goal.get("columna"))
        else None,
        minimax_depth=depth if _is_int(depth) else None
        algorithm=_text(prueba.get("algoritmo")),
        heuristic=_text(prueba.get("heuristica")),
        beam_width=prueba["k"] if _is_int(prueba.get("k")) else None,
        action_order=_text(prueba.get("orden_acciones")),
    )


# Función auxiliar para convertir un valor a texto si es una cadena
def _text(value: object) -> str | None:
    return value if isinstance(value, str) else None
```

B.6 Módulo experiments/verifier.py

```
"""Solution verifier (RF-42): checks 1 to 5 for single-unit searches, check 6 for Minimax."""

import math
from dataclasses import dataclass

from tactical_grid.algorithms.adversarial import AdversarialResult
from tactical_grid.algorithms.adversarial_dynamic import DynamicAdversarialResult
from tactical_grid.algorithms.common import PathProblem, SearchResult
from tactical_grid.domain import dynamic
```

```
10 from tactical_grid.domain.model import Board, GameState, Position
11 from tactical_grid.domain.rules import legal_actions
12
13
14 # Resultado de un chequeo individual: nombre, si pasó y un detalle
15 @dataclass(frozen=True)
16 class Check
17     name: str
18     passed: bool
19     detail: str
20
21
22 # Conjunto de chequeos hechos al resultado de un algoritmo
23 @dataclass(frozen=True)
24 class VerificationReport
25     algorithm: str
26     checks: tuple[Check, ...]
27
28     # El reporte está bien solo si todos los chequeos pasaron
29     @property
30     def ok(self) -> bool:
31         return all(check.passed for check in self.checks)
32
33
34 # Verifica un resultado de búsqueda repitiendo su camino con las reglas del
   ↪ dominio (chequeos 1 a 5)
35 def verify_search(problem: PathProblem, result: SearchResult) ->
   ↪ VerificationReport:
36     """Replay the reported path on the domain rules and run RF-42 checks 1
       ↪ to 5.
37
38     Purpose: independent check that a search result is a legal, consistent
       ↪ solution.
39     Preconditions: `problem` is the one the algorithm solved.
40     Postconditions: never raises on a malformed path; each check reports why
       ↪ it failed.
41         A result without solution has no path to verify and yields an empty
           ↪ report.
42     Complexity: O(L * U) for a path of L cells and U units.
43     AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function from RF-42.
44     AI intervention: none after the draft.
45     Student validation: tests/test_verifier.py checks that every algorithm's
       ↪ result passes and
46         that a teleport, a path through a wall, a wrong cost and a path not
           ↪ ending at the goal are
47         rejected.
48     """
49     # sin solución no hay camino que verificar
50     if not result.success:
51         return VerificationReport(result.algorithm, ())
```

```
52
53      board = problem.board
54      start = problem.initial.position_of(problem.unit_id)
55      path = result.path
56
57      # estados y costo que salen de repetir el camino paso a paso
58      states: list[GameState] = [problem.initial]
59      replay_cost = 0.0
60      bad_move: str | None = None
61      if not path or path[0] != start:
62          bad_move = f"path does not start at the unit's cell {start}"
63      else:
64          for step, cell in enumerate(path[1:], start=1):
65              # se busca entre los sucesores uno que lleve a la siguiente
                ↪ celda del camino
66              match = [
67                  (nxt, cost)
68                  for nxt, cost in problem.expand(states[-1])
69                  if nxt.position_of(problem.unit_id) == cell
70              ]
71              # si no hay, el movimiento es ilegal y se corta la repetición
72              if not match:
73                  bad_move = f"step {step}:
                    ↪ {states[-1].position_of(problem.unit_id)} -> {cell}"
74                  break
75              states.append(match[0][0])
76              replay_cost += match[0][1]
77
78      # celdas del camino que no son transitables
79      walls = [cell for cell in path if not board.is_passable(cell)]
80      # pasos cuyo estado es inválido (unidades superpuestas o sobre
        ↪ obstáculos)
81      invalid = [i for i, state in enumerate(states) if not
        ↪ _valid_state(problem, state)]
82      reported = result.cost
83      final = path[-1] if path else None
84
85      # los 5 chequeos: movimientos, obstáculos, costo, meta y estados válidos
86      checks = (
87          Check(
88              "valid moves",
89              bad_move is None,
90              "every step is a legal action" if bad_move is None else
                ↪ f"illegal move at {bad_move}",
91          ),
92          Check(
93              "no obstacles",
94              not walls,
95              "no cell is blocked" if not walls else f"blocked cells in path:
                ↪ {walls}",
```

```
96              ),
97              Check(
98                  "cost matches",
99                  bad_move is None and math.isclose(replay_cost, reported),
100                 f"reported {reported}, replayed {replay_cost}",
101             ),
102             Check(
103                 "goal reached",
104                 final == problem.goal,
105                 f"final cell {final}, goal {problem.goal}",
106             ),
107             Check(
108                 "no invalid states",
109                 not invalid,
110                 "all replayed states are valid"
111                 if not invalid
112                 else f"invalid states at steps {invalid}",
113             ),
114         )
115     return VerificationReport(result.algorithm, checks)
116
117
118 # Un estado es válido si no hay dos unidades en la misma celda y todas están
    ↪ en celdas transitables
119 def _valid_state(problem: PathProblem, state: GameState) -> bool:
120     cells: list[Position] = [pos for _, pos in state.positions]
121     # celdas repetidas significan unidades superpuestas
122     if len(set(cells)) != len(cells):
123         return False
124     return all(problem.board.is_passable(cell) for cell in cells)
125
126
127 # Verifica que toda acción que consideró Minimax sea legal (chequeo 6)
128 def verify_adversarial(
129     board: Board, state: GameState, result: AdversarialResult
130 ) -> VerificationReport:
131     """RF-42 check 6: every action Minimax considered is legal under the
        ↪ game rules.
132
133     Purpose: independent check that the adversarial search only expands
        ↪ legal actions.
134     Preconditions: `result` was produced from `state` with `record=True`, so
        ↪ that
135         `result.considered` lists every (state, action) pair the search
            ↪ tried.
136     Postconditions: never raises; also checks that the selected action was
        ↪ among the legal
137         root actions. A result without recorded actions fails the first
            ↪ check.
138     Complexity: O(N * U^2) for N recorded actions and U units.
```

```
    AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function from RF-42.
    AI intervention: none after the draft.
    Student validation: tests/test_adversarial.py checks that it rejects an illegal action and
        requires recorded actions.
    """
    # posiciones de las acciones consideradas que no eran legales en su estado
    illegal = [
        index
        for index, (node, action) in enumerate(result.considered)
        if action not in legal_actions(board, node)
    ]
    # si no se registró nada (y había acción) no se puede comprobar
    recorded = bool(result.considered) or result.action is None
    # la acción elegida debe estar entre las legales de la raíz
    root_legal = result.action is None or result.action in legal_actions(board, state)
    checks = (
        Check(
            "legal actions",
            recorded and not illegal,
            "every considered action is legal"
            if recorded and not illegal
            else (
                f"illegal actions at positions {illegal[:5]}"
                if illegal
                else "no considered actions were recorded (run with record=True)"
            ),
        ),
        Check(
            "selected action legal",
            root_legal,
            f"selected {result.action}",
        ),
    )
    return VerificationReport(result.algorithm, checks)


# Verifica que toda acción que consideró la búsqueda dinámica sea legal con sus reglas (chequeo 6)
def verify_adversarial_dynamic(
    board: Board, state: dynamic.DynamicState, result: DynamicAdversarialResult
) -> VerificationReport:
    """RF-42 check 6 for the dynamic variant: every considered action is legal.

    Purpose: independent check that the search never moves a unit in recovery or steals from an
```

```
182         immune carrier.
183     Preconditions: `result` was produced from `state` with `record=True`.
184     Postconditions: never raises; also checks that the selected action was
        ↪ legal at the root.
185     Complexity: O(N * U^2) for N recorded actions and U units.
186     AI usage: Yes. Claude Code (Sonnet 5.5) drafted the function from the
        ↪ approved plan.
187     AI intervention: none after the draft.
188     Student validation: tests/test_adversarial_dynamic.py checks that it
        ↪ accepts the search and
189         rejects a move of a unit in recovery.
190     """
191     illegal = [
192         index
193         for index, (node, action) in enumerate(result.considered)
194         if action not in dynamic.legal_actions(board, node)
195     ]
196     recorded = bool(result.considered) or result.action is None
197     root_legal = result.action is None or result.action in
        ↪ dynamic.legal_actions(board, state)
198     checks = (
199         Check(
200             "legal actions",
201             recorded and not illegal,
202             "every considered action is legal"
203             if recorded and not illegal
204             else (
205                 f"illegal actions at positions {illegal[:5]}"
206                 if illegal
207                 else "no considered actions were recorded (run with
                    ↪ record=True)"
208             ),
209         ),
210         Check("selected action legal", root_legal, f"selected
            ↪ {result.action}"),
211     )
212     return VerificationReport(result.algorithm, checks)
```

Apéndice C

Resultados Completos (CSV/JSON)

Los archivos CSV y JSON completos de todos los experimentos se encuentran en `docs/results/`.